WELLNESS STUDIO OS

Database Design Document

PostgreSQL Application Database · ERP Data Model · AI, Agent and Automation Data

Item	Detail
Document	Document 4 of 6 — Database Design Document
Version / Date	1.0 · 29 September 2026
Sources of truth	Master PRD v1.0 (incl. Appendix A conflict register) · Solution Architecture Document v1.0 · Module Specification v1.0
Scope	Database design only. PRD content, architecture and API contracts are not restated.
Target platform	PostgreSQL 16+ (application DB, Prisma migrations per rebuild plan) · ERPNext/Frappe on MariaDB (system of record, upstream untouched)
Audience	6-member development team, QA, ERP, AI and automation engineers
Status	Implementation-ready draft for CTO / mentor review

How to read this document. Because the architecture makes the ERP the system of record for business facts (PRD Part E, Architecture §6.2/§7.2), the physical design has two parts: **(A) the PostgreSQL application database** (tenants, identity, configuration, outbox, notification, AI, agent, audit, reporting) with full DDL-level column definitions, and **(B) the ERP data model** (Frappe DocTypes owned by the custom app *wellness_studio*) specified with the same rigour. The application database never stores a second copy of ERP business truth; it stores ERP document IDs only.

Conventions. *Priority: P0 = mandatory for 5-Day MVP; P1 = important (Phase 1 additions / first sprint after MVP); P2 = Phase 2 or later. Category tags follow Section 2. "Zero-uuid" in a unique expression means COALESCE(col, '00000000-0000-0000-0000-000000000000'). Every PostgreSQL table lists all columns including the standard tenant/timestamp columns.*

Source conflicts resolved (recorded so the team does not re-litigate).

#	Conflict	Resolution used in this document
D-01	Module Spec lists Notification Log as an ERP DocType; Architecture §6.2 and PRD Part E place notification records and delivery status in the application DB.	Application DB (notify schema). Architecture and PRD take precedence; ERP holds no notification records.
D-02	Module Spec lists Tenant, Studio, Branch as ERP DocTypes (P0); Architecture §6.2 makes the App DB the owner of tenants/studios/branches.	App DB is authoritative. ERP keeps read-only mirror DocTypes (Wellness Tenant/Studio/Branch Scope) written only by the onboarding routine, solely to support permission hooks and links.
D-03	PRD B2B-SET-01 lists event categories in the Configuration Center; Architecture says plans/class types (business records) stay in ERP, presentation config in App DB.	Plans and class types in ERP; event categories in App DB (config.event_categories); ERP Event stores the category code, validated through the adapter.
D-04	Architecture leaves ERP tenant isolation mechanism open (Company + hooks vs site-per-tenant; spike S2).	Design supports both: every DocType carries a tenant link; platform.tenant_erp_routes maps tenant → site/company.
D-05	Feedback records: Architecture says App DB referencing ERP session IDs; Module Spec says stored "against session and class type".	App DB (ai.ai_feedback_submissions / ai_feedback_analysis) with ERP session reference; raw text and classification stored separately as Module Spec requires.

Table of Contents

1 Database Overview

2 Entity Inventory

3 Conceptual Data Model

4 Logical Database Design

5 Physical Database Design (5A PostgreSQL · 5B ERP DocTypes)

6 Multi-Tenant Database Strategy

7 ERP Data Model

8 AI Data Model

9 Agent Data Model

10 Automation Data Model

11 Audit & Compliance Tables

12 Indexing Strategy

13 Data Retention & Archival

14 Database Security

15 MVP Database Scope

Appendix A Implementation DDL patterns (RLS, partitioning, immutability, outbox)

Appendix B Migration order and team ownership map

1. Database Overview

1.1 Database Objectives

ID	Objective	Measure
DBO-1	Give every tenant an isolated, branch-ready data foundation from day one (NFR-SCL-01, BR-GEN-02).	Every table carries tenant_id; RLS on all tenant tables; cross-tenant test suite passes.
DBO-2	Keep exactly one source of truth per business fact: ERP for business records, PostgreSQL for platform/identity/config/audit/AI/notification data.	No table stores an ERP-owned business field beyond an ID reference or a labelled audit/display snapshot.
DBO-3	Make every consequential action traceable (NFR-AUD-01..04).	Append-only audit with correlation ID linking UI → API → ERP → n8n → WhatsApp.
DBO-4	Support reliable side effects: commit first, notify after, idempotent consumers (NFR-REL-01/02).	Outbox rows are written in the same transaction as audit; duplicate events yield one message.
DBO-5	Be AI-ready and agent-ready without schema redesign (AD08/AD09, PRD §11).	Agent tables reserved; AI audit, pending actions and usage logs exist in Phase 1.
DBO-6	Enable parallel development by six engineers with minimal merge conflicts.	Schema-per-bounded-context, one migration folder per schema, single owner per schema (Appendix B).
DBO-7	Scale by partitioning and replicas rather than redesign (§6.4 architecture).	High-volume tables are range-partitioned by time from the first migration.

1.2 Database Design Principles

- ERP is truth, PostgreSQL is platform. If losing a record would corrupt a business fact it lives in the ERP; if it governs who may do what, how something is delivered, or how it is presented, it lives in PostgreSQL (Architecture §7.2 rule of thumb).
- Reference by ID, never replicate. App DB tables hold ERP docnames (e.g. MEM-00042) as varchar references. There is no foreign-key across stores; integrity is enforced by the adapter and the nightly reconciliation job (ops.reconciliation_*).
- Tenant column on every table + row-level security keyed to a per-request setting (defence in depth on top of the tenant guard).
- Schema per bounded context: platform, iam, config, ops, notify, ai, agent, audit, reporting. Each schema is owned by one team member and migrated independently.
- UUID primary keys (gen_random_uuid(); UUIDv7 when available) for App DB rows so IDs are non-guessable, generated by the system and never supplied by clients (BR-GEN-04). ERP documents use human-readable naming series (MEM-NNNNN).
- Timestamps are timestamptz in UTC; business-time display uses tenant/branch time zone (default Asia/Kolkata). Money is numeric(14,2) in INR (A-07); currency column present for future use.
- Append-only for evidence: audit, AI tool-call audit, delivery events, consent history, correction history. The application role has no UPDATE/DELETE grant on these tables.
- Archive, do not destroy, critical history (BR-GEN-03): soft status/archived flags for configuration and identities; time-based partition detach for logs.
- Configure, do not hard-code (BR-GEN-05): defaults (capacity 30, cancellation cutoff 240 min, reminder offsets) are rows in config.tenant_settings / config.notification_rules, seeded by the onboarding routine.
- Expand → migrate → contract migrations only, so rollbacks remain backward-compatible (Architecture §13.3).
- No secrets in the database: credentials are pointers into the secret store (secret_ref, credential_ref). OTPs, refresh tokens and recipient identifiers are stored as hashes or envelope-encrypted values.
- Constraints in the database, not only in code: CHECK, UNIQUE and FK constraints encode the rules that must hold even if a code path is missed.

1.3 Multi-Tenant Strategy (summary — detail in Section 6)

Chosen model: shared database, shared schema, tenant_id discriminator on every row, enforced by PostgreSQL row-level security with a per-request setting (SET LOCAL app.tenant_id). The hierarchy is Tenant → Studio → Branch. The ERP side uses one Frappe site for beta with each tenant as an ERPNext Company; the routing table allows a noisy or enterprise tenant to be moved to a dedicated site with no caller change.

1.4 Data Ownership Strategy

Owner	Data	Store
ERP (custom app + ERPNext)	Members, memberships, membership plans, class types, schedules, sessions, bookings, attendance (+ correction history), events and registrations, leads, payment records, invoices, trainer/staff business records	ERPNext DB (Frappe DocTypes)

Owner	Data	Store
Application DB — platform	Tenants, studios, branches, tiers, entitlements, feature flags, domains, ERP routes	PostgreSQL: platform
Application DB — identity	Users, credentials, auth identities, sessions, OTP, roles, permissions, grants, service principals, user↔ERP links	PostgreSQL: iam
Application DB — configuration	Branding, settings, event categories, notification templates and rules, FAQ, report definitions	PostgreSQL: config, reporting
Application DB — operational	Outbox, idempotency keys, webhook inbox, reconciliation	PostgreSQL: ops
Application DB — automation	Notification log, WhatsApp/email records, delivery events, consents, workflow executions, alerts	PostgreSQL: notify
Application DB — AI / Agent	Sessions, conversations, pending actions, AI audit, recommendations, feedback, usage, evaluation; agent registry/tasks/memory	PostgreSQL: ai, agent
Application DB — audit	Audit log, change history, activity, security events, retention policy	PostgreSQL: audit
Repository (versioned as code)	n8n workflow definitions, prompt template files, permission catalogue seeds	Git

1.5 Scalability Considerations

- Row volume drivers: audit_logs, notification_messages, whatsapp_messages, outbox_events, workflow_executions, ai_tool_call_audit, ai_usage_logs, security_events, activity_events. All are declared with PARTITION BY RANGE on their time column (monthly) so old partitions can be detached, archived and dropped cheaply.
- Read scaling: managed PostgreSQL with read replica for reporting and audit views; reporting.kpi_snapshots pre-aggregates ERP KPIs so dashboards meet the 3-second target (NFR-PRF-02).
- Hot paths never touch large tables: booking/check-in/payment initiation go to the ERP atomic methods; the App DB write on that path is one audit row + one outbox row.
- Connection strategy: pooled connections (PgBouncer transaction mode). RLS context is set with SET LOCAL inside the request transaction so it is safe with transaction pooling.
- Tenant growth: shared schema comfortably serves the pilot and early tenants; tenant-hash partitioning of the largest tables, and finally schema-/database-per-tenant for an enterprise customer, are documented triggers in Section 6.5 — the tenant_id column and routing table mean no application change.
- ERP scaling: adapter routes by tenant → site (platform.tenant_erp_routes); heavy reports move to scheduled aggregates in reporting.kpi_snapshots.

2. Entity Inventory

Classification uses the six required classes. Stores: [ERP] = ERPNext custom app DocType; [PG] = PostgreSQL application DB. “Ph” is the first phase in which the entity is needed (P0 MVP, P1, P2).

2.1 Core Entities

Entity	Module(s)	Store	Ph
Tenant, Studio, Branch	Tenant Management (B2B-14)	PG platform (+ERP mirror scope)	P0
User, Auth Identity, Auth Session	Auth, Staff, Profile	PG iam	P0
Role, Permission, Role-Permission, Role Assignment, Permission Grant	Staff Management, RBAC	PG iam	P0
Member	Members (B2B-02), B2C Profile	ERP	P0
Lead (Workshop Registrant)	CRM (B2B-07)	ERP	P0
Wellness Staff (trainer)	Staff Management	ERP (+PG iam identity)	P0
Membership Plan, Membership	Memberships (B2B-03)	ERP	P0
Class Type, Class Schedule, Class Session	Classes (B2B-04)	ERP	P0
Event	Events (B2B-08)	ERP	P0

2.2 Transactional Entities

Entity	Module(s)	Store	Ph
Booking (incl. waitlist position)	Bookings	ERP	P0
Attendance Record	Attendance	ERP	P0
Event Registration	Events	ERP	P0
Payment Record	Payments	ERP	P0
Outbox Event, Idempotency Key, Webhook Inbox	Platform reliability	PG ops	P0
Notification Message, WhatsApp Message, Delivery Event	Notifications	PG notify	P0
Workflow Execution, Admin Alert	Automation	PG notify	P0
Communication Consent	Notifications / Profile	PG notify	P1
KPI Snapshot, Report Export	Reports	PG reporting	P1 / P2
Reconciliation Run / Finding	Payments, ERP sync	PG ops	P1
Email Log, Suppression List	Email channel	PG notify	P2

2.3 Configuration Entities

Entity	Store	Ph
Subscription Tier, Tier Feature, Tenant Feature Flag, Tenant Domain	PG platform	P0
Tenant Branding, Tenant Settings, Event Category	PG config	P0
Notification Template, Notification Rule	PG config	P0
FAQ Entry (approved knowledge)	PG config	P1
Report Definition	PG reporting	P1
Tenant ERP Route, ERP Contract Version	PG platform / ops	P1 / P2
AI Prompt Version	PG ai	P1
Data Retention Policy	PG audit	P1
Membership Plan, Class Type, Class Schedule (business configuration)	ERP	P0

2.4 AI Entities

Entity	Store	Ph
AI Session, AI Conversation (turns)	PG ai	P0
AI Pending Action (propose → confirm)	PG ai	P0
AI Tool Call Audit	PG ai	P0
AI Recommendation	PG ai	P1
AI Feedback Submission, AI Feedback Analysis, Feedback Theme Summary	PG ai	P1
AI Usage Log, AI Eval Run	PG ai	P1

2.5 Agent Entities

Entity	Store	Ph
Agent Registry, Agent Tenant Config, Agent Tool Grant	PG agent	P2

Entity	Store	Ph
Agent Task, Agent Execution	PG agent	P2
Agent Decision (approval queue)	PG agent	P2
Agent Memory, Agent Log	PG agent	P2
Service Principal (agent/worker identity)	PG iam	P1

2.6 Audit Entities

Entity	Store	Ph
Audit Log	PG audit	P0
Security Event	PG audit	P0
Change History, Activity Event	PG audit	P1
Automation Audit Trail	PG notify	P1
AI Tool Call Audit (AI-specific audit)	PG ai	P0
Attendance Correction (immutable child)	ERP	P0
Data Subject Request	PG audit	P2

2.7 Entity-to-module traceability

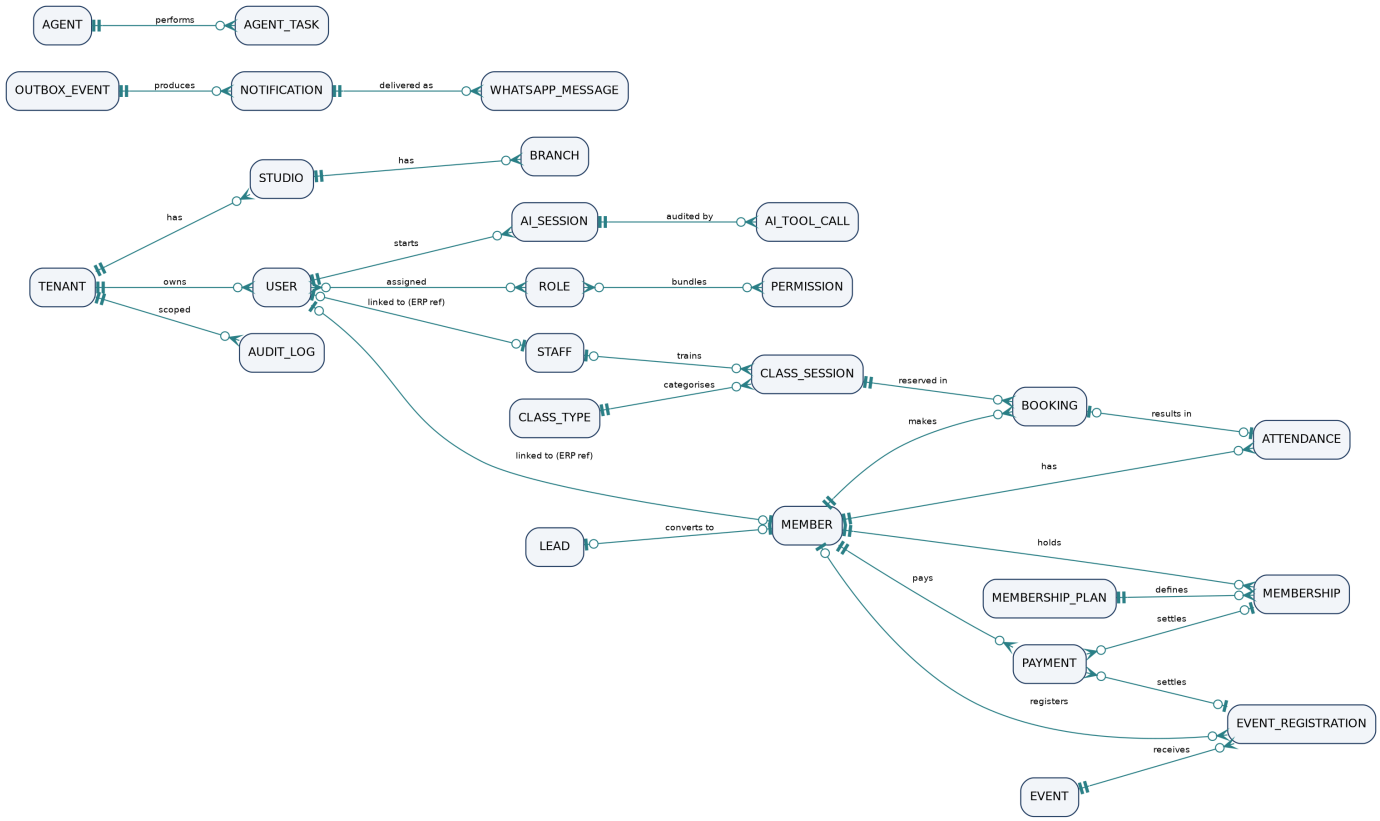
Module Spec module	Primary entities
Dashboard / Reports	reporting.report_definitions, reporting.kpi_snapshots (aggregates over ERP entities)
CRM / Leads	ERP Lead, Lead Activity; notify.notification_messages (confirmation)
Members	ERP Member; iam.user_erp_links
Memberships	ERP Membership Plan, Plan Class Entitlement, Membership
Classes	ERP Class Type, Class Schedule, Class Session
Bookings	ERP Booking
Attendance	ERP Attendance Record, Attendance Correction
Events	ERP Event, Event Registration; config.event_categories
Payments	ERP Payment Record; ops.webhook_inbox; ops.idempotency_keys
Staff Management	ERP Wellness Staff; iam.users/roles/grants
Settings	config.*; platform.tenant_feature_flags
Notifications / Automation	notify.*, ops.outbox_events, config.notification_*
Tenant Management	platform.*
AI Assistants	ai.* ; config.faq_entries
Agents	agent.*

3. Conceptual Data Model

3.1 High-Level ER Diagram

Entities are shown at concept level. ERP-owned entities (Member ... Payment) live in the ERP; relations between App DB rows and ERP entities are by-ID references, not database foreign keys. Rendered diagrams are followed by the Mermaid source so the team can paste them into repository docs.

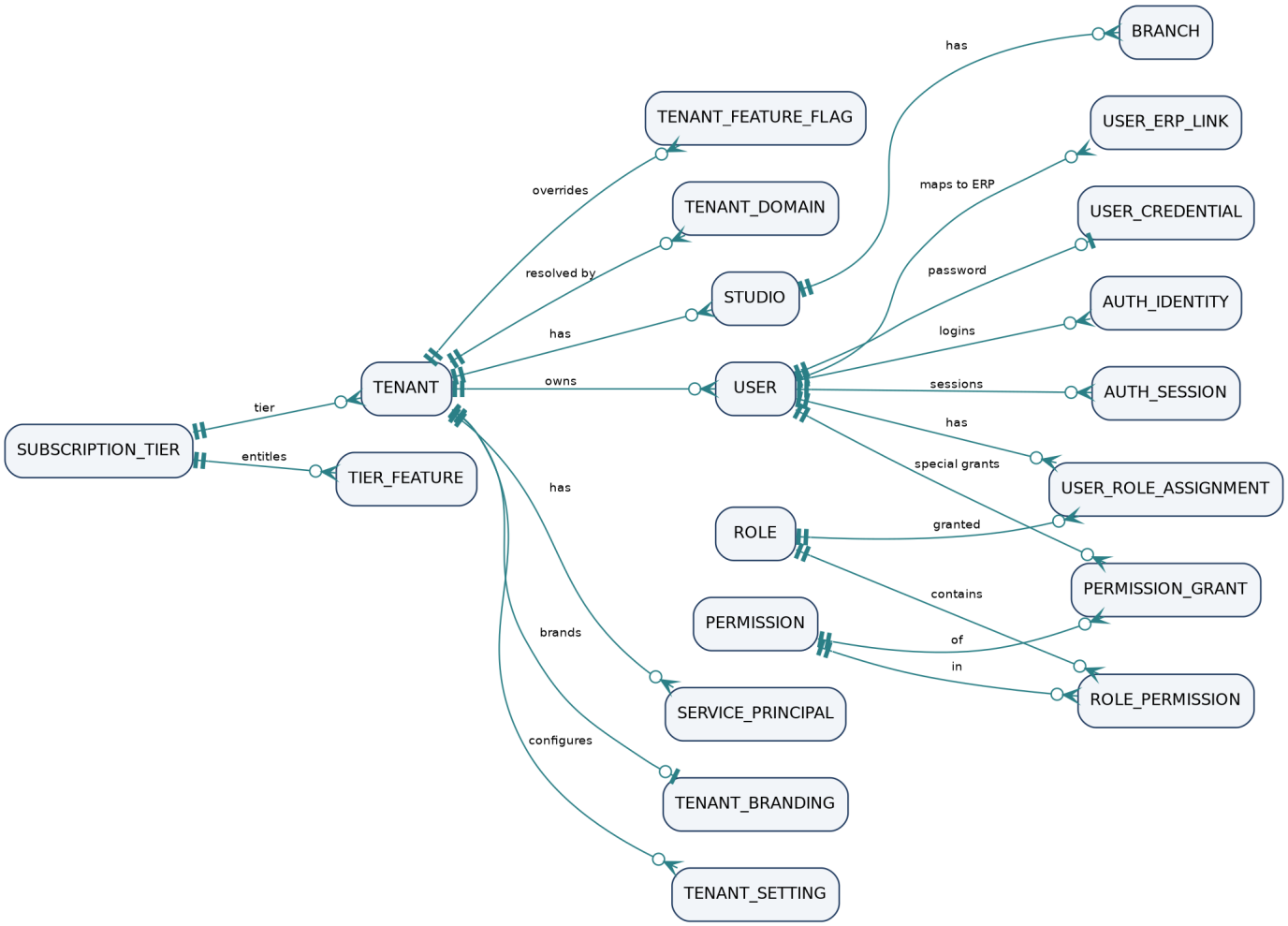
Figure 1 — Domain-level conceptual model (App DB and ERP)




```
erDiagram
    TENANT ||--o{ STUDIO : "has"
    STUDIO ||--o{ BRANCH : "has"
    TENANT ||--o{ USER : "owns"
    USER }o--o{ ROLE : "assigned"
    ROLE }o--o{ PERMISSION : "bundles"
    USER |o--o| MEMBER : "linked to (ERP ref)"
    USER |o--o| STAFF : "linked to (ERP ref)"
    LEAD |o--o| MEMBER : "converts to"
    MEMBER ||--o{ MEMBERSHIP : "holds"
    MEMBERSHIP_PLAN ||--o{ MEMBERSHIP : "defines"
    CLASS_TYPE ||--o{ CLASS_SESSION : "categorises"
    STAFF |o--o{ CLASS_SESSION : "trains"
    MEMBER ||--o{ BOOKING : "makes"
    CLASS_SESSION ||--o{ BOOKING : "reserved in"
    BOOKING |o--o| ATTENDANCE : "results in"
    MEMBER ||--o{ ATTENDANCE : "has"
    EVENT ||--o{ EVENT_REGISTRATION : "receives"
    MEMBER |o--o{ EVENT_REGISTRATION : "registers"
    MEMBER ||--o{ PAYMENT : "pays"
    PAYMENT }o--o| MEMBERSHIP : "settles"
    PAYMENT }o--o| EVENT_REGISTRATION : "settles"
    OUTBOX_EVENT ||--o{ NOTIFICATION : "produces"
    NOTIFICATION ||--o{ WHATSAPP_MESSAGE : "delivered as"
    USER ||--o{ AI_SESSION : "starts"
    AI_SESSION ||--o{ AI_TOOL_CALL : "audited by"
    AGENT ||--o{ AGENT_TASK : "performs"
    TENANT ||--o{ AUDIT_LOG : "scoped"
```

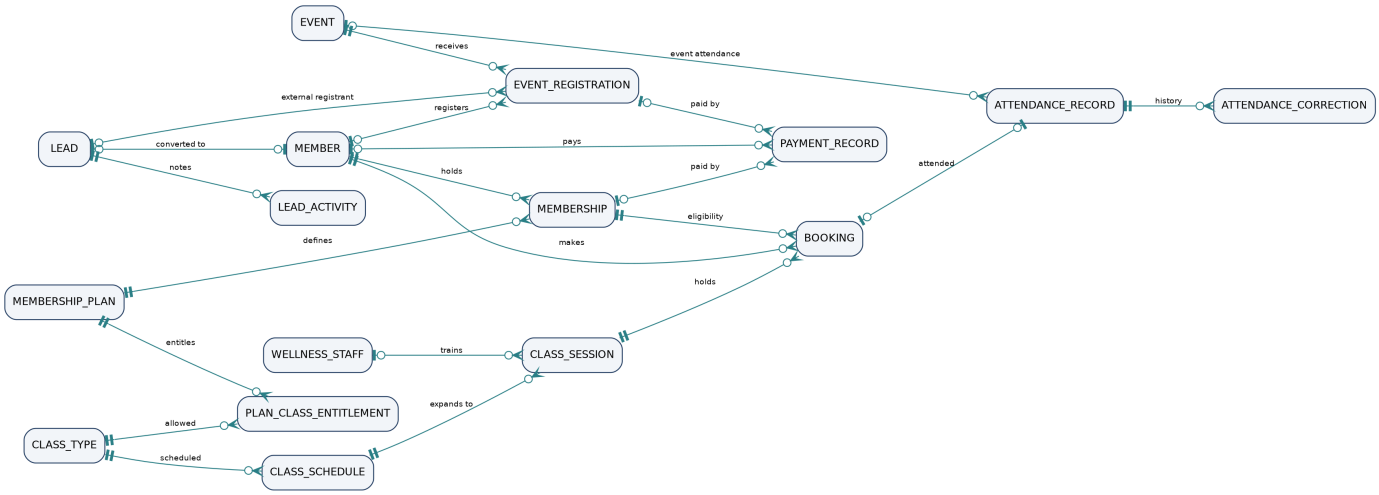
3.2 Detailed conceptual views

Figure 2 — Tenant, identity and access model (PostgreSQL)



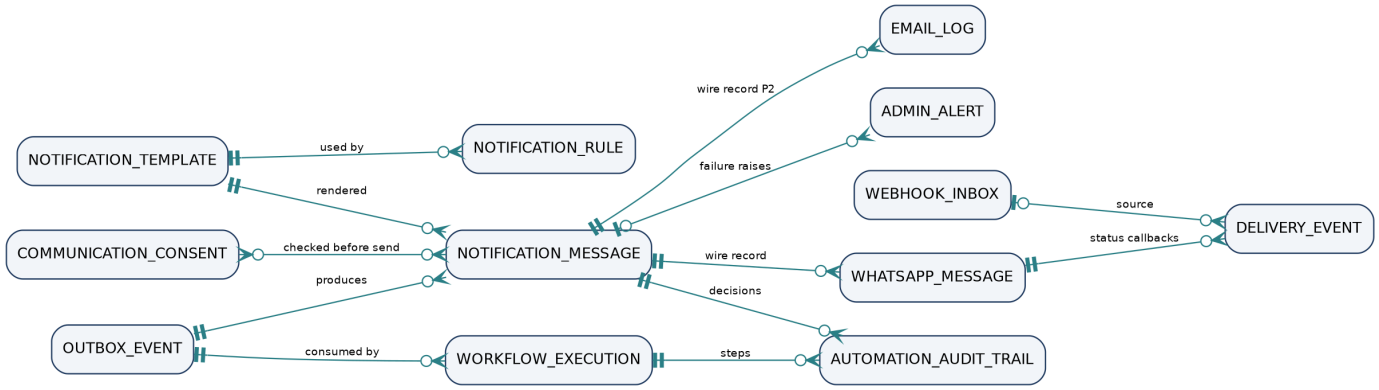

```
erDiagram
    SUBSCRIPTION_TIER ||--o{ TENANT : "tier"
    SUBSCRIPTION_TIER ||--o{ TIER_FEATURE : "entitles"
    TENANT ||--o{ TENANT_FEATURE_FLAG : "overrides"
    TENANT ||--o{ TENANT_DOMAIN : "resolved by"
    TENANT ||--o{ STUDIO : "has"
    STUDIO ||--o{ BRANCH : "has"
    TENANT ||--o{ USER : "owns"
    USER ||--o| USER_CREDENTIAL : "password"
    USER ||--o{ AUTH_IDENTITY : "logins"
    USER ||--o{ AUTH_SESSION : "sessions"
    USER ||--o{ USER_ROLE_ASSIGNMENT : "has"
    ROLE ||--o{ USER_ROLE_ASSIGNMENT : "granted"
    ROLE ||--o{ ROLE_PERMISSION : "contains"
    PERMISSION ||--o{ ROLE_PERMISSION : "in"
    USER ||--o{ PERMISSION_GRANT : "special grants"
    PERMISSION ||--o{ PERMISSION_GRANT : "of"
    USER ||--o{ USER_ERP_LINK : "maps to ERP"
    TENANT ||--o{ SERVICE_PRINCIPAL : "has"
    TENANT ||--o| TENANT_BRANDING : "brands"
    TENANT ||--o{ TENANT_SETTING : "configures"
```

Figure 3 — ERP business data model (ERPNext custom app DocTypes)



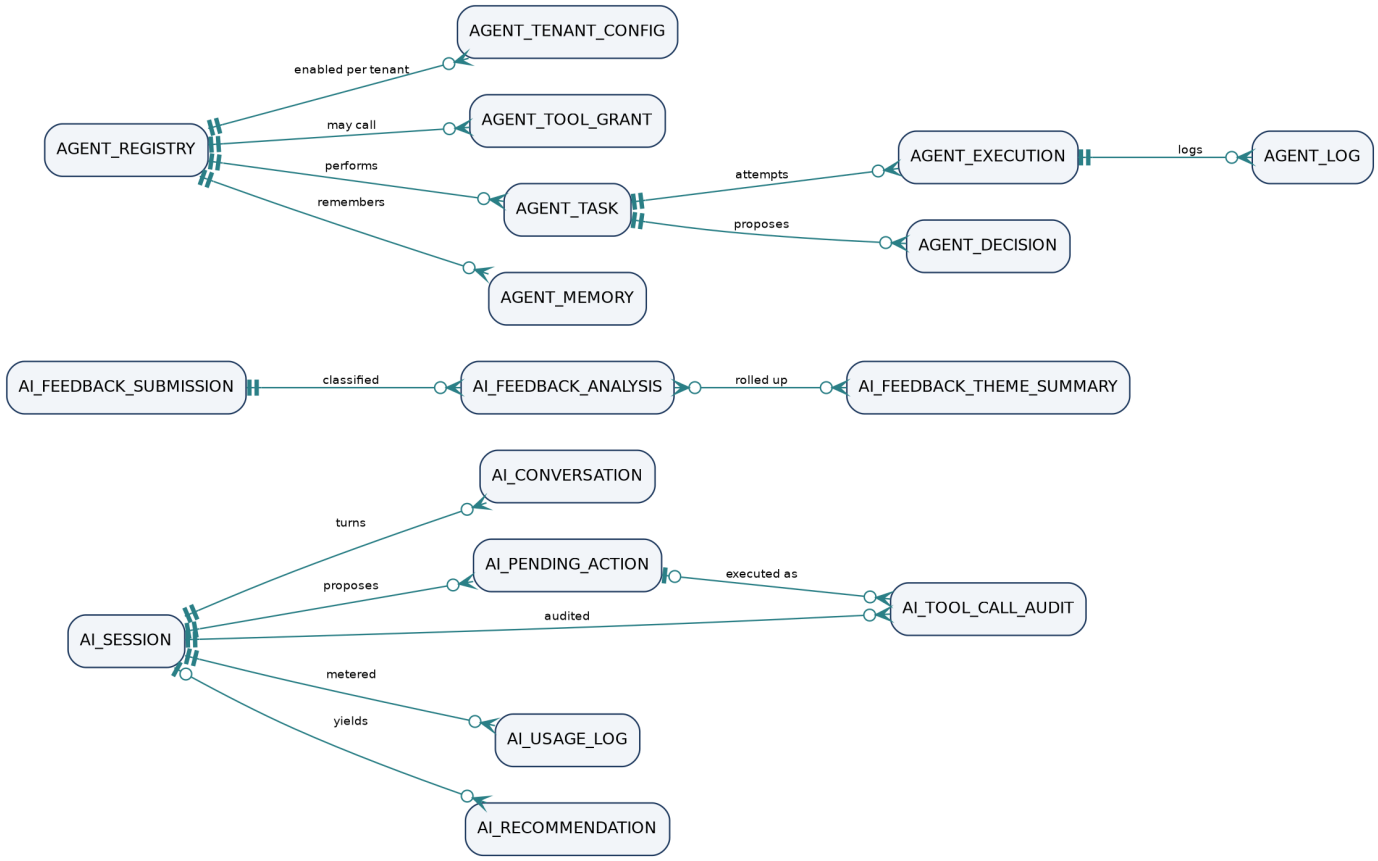

```
erDiagram
    MEMBER ||--o{ MEMBERSHIP : "holds"
    MEMBERSHIP_PLAN ||--o{ MEMBERSHIP : "defines"
    MEMBERSHIP_PLAN ||--o{ PLAN_CLASS_ENTITLEMENT : "entitles"
    CLASS_TYPE ||--o{ PLAN_CLASS_ENTITLEMENT : "allowed"
    CLASS_TYPE ||--o{ CLASS_SCHEDULE : "scheduled"
    CLASS_SCHEDULE ||--o{ CLASS_SESSION : "expands to"
    WELLNESS_STAFF |o--o{ CLASS_SESSION : "trains"
    MEMBER ||--o{ BOOKING : "makes"
    CLASS_SESSION ||--o{ BOOKING : "holds"
    MEMBERSHIP ||--o{ BOOKING : "eligibility"
    BOOKING |o--o| ATTENDANCE_RECORD : "attended"
    ATTENDANCE_RECORD ||--o{ ATTENDANCE_CORRECTION : "history"
    EVENT ||--o{ EVENT_REGISTRATION : "receives"
    MEMBER |o--o{ EVENT_REGISTRATION : "registers"
    LEAD |o--o{ EVENT_REGISTRATION : "external registrant"
    EVENT |o--o{ ATTENDANCE_RECORD : "event attendance"
    LEAD ||--o{ LEAD_ACTIVITY : "notes"
    LEAD |o--o| MEMBER : "converted to"
    MEMBER |o--o{ PAYMENT_RECORD : "pays"
    MEMBERSHIP |o--o{ PAYMENT_RECORD : "paid by"
    EVENT_REGISTRATION |o--o{ PAYMENT_RECORD : "paid by"
```

Figure 4 — Automation, notification and event model (PostgreSQL)




```
erDiagram
    OUTBOX_EVENT ||--o{ WORKFLOW_EXECUTION : "consumed by"
    OUTBOX_EVENT ||--o{ NOTIFICATION_MESSAGE : "produces"
    NOTIFICATION_TEMPLATE ||--o{ NOTIFICATION_RULE : "used by"
    NOTIFICATION_TEMPLATE ||--o{ NOTIFICATION_MESSAGE : "rendered"
    NOTIFICATION_MESSAGE ||--o{ WHATSAPP_MESSAGE : "wire record"
    NOTIFICATION_MESSAGE ||--o{ EMAIL_LOG : "wire record P2"
    WHATSAPP_MESSAGE ||--o{ DELIVERY_EVENT : "status callbacks"
    WEBHOOK_INBOX |o--o{ DELIVERY_EVENT : "source"
    WORKFLOW_EXECUTION ||--o{ AUTOMATION_AUDIT_TRAIL : "steps"
    NOTIFICATION_MESSAGE ||--o{ AUTOMATION_AUDIT_TRAIL : "decisions"
    COMMUNICATION_CONSENT }o--o{ NOTIFICATION_MESSAGE : "checked before send"
    NOTIFICATION_MESSAGE |o--o{ ADMIN_ALERT : "failure raises"
```

Figure 5 — AI and agent model (PostgreSQL)




```
erDiagram
    AI_SESSION ||--o{ AI_CONVERSATION : "turns"
    AI_SESSION ||--o{ AI_PENDING_ACTION : "proposes"
    AI_SESSION ||--o{ AI_TOOL_CALL_AUDIT : "audited"
    AI_PENDING_ACTION |o--o{ AI_TOOL_CALL_AUDIT : "executed as"
    AI_SESSION ||--o{ AI_USAGE_LOG : "metered"
    AI_SESSION |o--o{ AI_RECOMMENDATION : "yields"
    AI_FEEDBACK_SUBMISSION ||--o{ AI_FEEDBACK_ANALYSIS : "classified"
    AI_FEEDBACK_ANALYSIS }o--o{ AI_FEEDBACK_THEME_SUMMARY : "rolled up"
    AGENT_REGISTRY ||--o{ AGENT_TENANT_CONFIG : "enabled per tenant"
    AGENT_REGISTRY ||--o{ AGENT_TOOL_GRANT : "may call"
    AGENT_REGISTRY ||--o{ AGENT_TASK : "performs"
    AGENT_TASK ||--o{ AGENT_EXECUTION : "attempts"
    AGENT_TASK ||--o{ AGENT_DECISION : "proposes"
    AGENT_REGISTRY ||--o{ AGENT_MEMORY : "remembers"
    AGENT_EXECUTION ||--o{ AGENT_LOG : "logs"
```

3.3 Entity Relationships and Cardinality Rules

#	Relationship	Cardinality	Rule
R1	Tenant → Studio → Branch	1 : N : N	A studio belongs to exactly one tenant; a branch to exactly one studio (composite FK on tenant_id).
R2	Tenant → User	1 : N	Email and phone are unique per tenant (not globally).
R3	User ↔ Role	N : M via user_role_assignments	Assignment is scoped to studio/branch; NULL scope = wider scope. At most one active assignment per (user, role, scope).
R4	Role ↔ Permission	N : M	System roles are seeded; custom roles may only combine existing permissions.
R5	User → Member/Staff (ERP)	0..1 : 1 (per doctype)	A member user links to exactly one ERP Member; linking resolves on verified phone; never creates a duplicate Member.
R6	Lead → Member	0..1 : 0..1	Conversion sets Member.source_lead and Lead.converted_member; lead history retained (BR-CRM-02).
R7	Membership Plan → Membership	1 : N	Membership stores a plan snapshot; plan edits never change existing memberships (BR-MSP-06).
R8	Member → Membership	1 : N	States: Pending Approval, Active, Expired, Suspended, Cancelled; payment and approval independent (BR-MSP-02).
R9	Class Type → Class Session	1 : N	Class type archived not deleted once used.
R10	Class Session → Booking	1 : N (≤ capacity confirmed)	Confirmed bookings never exceed capacity; unique (member, session) among Booked/Waitlisted.
R11	Booking → Attendance Record	1 : 0..1	Attendance is per session; one record per member per session (BR-ATT-04). Attendance may exist without booking only if drop-in allowed.

#	Relationship	Cardinality	Rule
R12	Event → Event Registration	1 : N (≤ capacity confirmed)	Registrant is a member XOR a lead (Workshop Registrant).
R13	Payment Record → Membership | Event Registration	N : 0..1 each	A payment settles exactly one purpose; verified confirmation required for Paid.
R14	Outbox Event → Notification Message	1 : N	One event may address several recipients; unique dedupe key per (event, recipient, template).
R15	Notification Message → WhatsApp Message → Delivery Event	1 : N : N	Retries create additional wire records; delivery events append.
R16	AI Session → Conversation / Pending Action / Tool Call Audit	1 : N	Pending action executes only its stored arguments (hash verified).
R17	Agent → Task → Execution / Decision	1 : N : N	Agents act with a service principal scoped to one tenant and one domain toolset.
R18	Tenant → every tenant-scoped table	1 : N	Non-null tenant_id, RLS-enforced; audit and security events retained even if tenant closes.

4. Logical Database Design

The logical schema below covers the sixteen required entity groups. Each group maps to the physical tables in Section 5 (PostgreSQL) or Section 5B (ERP DocTypes).

4.1 Tenants

Aspect	Definition
Physical mapping	platform.tenants (+ studios, branches, tiers, flags, domains)
Description	The subscribing studio business and the root of isolation. Carries tier, timezone, currency, ERP company binding.
Purpose	Creates the isolation boundary and drives tier-based feature entitlement (Basic/Premium/Ultra).
Relationships	Tenant 1:N Studio 1:N Branch; Tenant 1:N User; Tenant 1:N every tenant-scoped record; N:1 Subscription Tier.
Business rules	Onboarding creates tenant → first studio → first branch → Owner user → default configuration (B2B-TEN-02). Tier flags gate AI/automation (BR-AI-11). Cross-tenant access is denied and logged (BR-GEN-02). Tier is separate from member plans (BR-MSP-01).

4.2 Users

Aspect	Definition
Physical mapping	iam.users, user_credentials, auth_identities, auth_sessions, otp_challenges, user_erp_links
Description	Application login identities for staff, members and platform operators. Members are not ERP users.
Purpose	Authentication, session management and mapping identity to ERP Member/Staff records.
Relationships	Tenant 1:N User; User 1:N Auth Identity/Session; User 0..1:1 ERP Member or Staff via user_erp_links; User N:M Role.
Business rules	Staff: email+password, mandatory reset, lockout, MFA before prod. Members: phone OTP primary, Google optional; linking on verified phone never duplicates a member. Deactivated users cannot authenticate; history keeps attribution.

4.3 Roles

Aspect	Definition
Physical mapping	iam.roles, user_role_assignments
Description	Named bundles of permissions: Owner, Admin, Front Desk, Trainer, Member, Visitor, Platform Administrator.
Purpose	Least-privilege access with scope by tenant/studio/branch.
Relationships	Role N:M Permission; Role N:M User (scoped).
Business rules	Six roles plus operator role. Owner: whole tenant; Admin: assigned studios/branches; Front Desk: assigned studio/branch; Trainer: assigned sessions only. Role changes take effect on the next request and are audited. Code never checks role names inline.

4.4 Permissions

Aspect	Definition
Physical mapping	iam.permissions, role_permissions, permission_grants
Description	Granular resource:action catalogue plus explicit special grants (e.g. bookings:on_behalf for trainers).
Purpose	Fine-grained authorisation and AI tool filtering (risk_class).
Relationships	Permission N:M Role; Permission 1:N Grant; Grant N:1 User.
Business rules	No role may bypass eligibility, payment or capacity rules (BR-BKG-06). Grants are scoped, revocable, audited; revoked rows retained. AI tools are filtered per user and re-checked at execution.

4.5 Leads

Aspect	Definition
Physical mapping	ERP Lead, Lead Activity
Description	Prospect or workshop registrant captured from forms, ads, walk-ins or the Enquiry Assistant.
Purpose	Measurable acquisition and clean prospect/member distinction.
Relationships	Lead 0..1:1 Member (on conversion); Lead 1:N Lead Activity; Lead 0..N Event Registration.
Business rules	A submission creates a Lead, never a Member (BR-CRM-01). Source immutable; campaign Admin-editable. Duplicates (phone/email) flagged or merged (BR-CRM-03). Conversion uses standard member creation; history retained. Messages subject to consent (BR-CRM-04).

4.6 Members

Aspect	Definition
Physical mapping	ERP Member
Description	The single record per person per tenant: identity, contact, status, emergency contact, optional health notes, photo.
Purpose	Single source of member truth for all modules.
Relationships	Member 1:N Membership, Booking, Attendance, Payment, Event Registration; 0..1 App User; 0..1 source Lead.
Business rules	ID system-generated MEM-NNNNN. Unique email and phone per tenant. Status active/inactive/frozen (+archived). Archive instead of delete; Front Desk cannot delete; Trainer read-only on assigned; Member edits only permitted own fields. All changes audited with before/after.

4.7 Membership Plans

Aspect	Definition
Physical mapping	ERP Membership Plan, Plan Class Entitlement
Description	Studio-defined commercial plans: price, validity, entitlements, booking limits, approval requirement.
Purpose	Configurable commercial model with no deployment.
Relationships	Plan 1:N Membership; Plan N:M Class Type via entitlement.
Business rules	Price ≥ 0; validity > 0; edits audited and never alter existing memberships (BR-MSP-06). Deactivate rather than delete when used. Separate from platform tiers.

4.8 Memberships

Aspect	Definition
Physical mapping	ERP Membership
Description	A member’s assignment to a plan with validity dates and a state machine.
Purpose	Drives eligibility for booking and renewal pipeline.
Relationships	Member 1:N Membership; Plan 1:N Membership; Membership 0..N Payment; Membership 1:N Booking (eligibility).
Business rules	States Pending Approval → Active → Expired / Suspended / Cancelled. Payment status and approval status independent — confirmed payment alone does not activate an approval-required membership. End > start; member must exist. Expiry set automatically by ERP scheduler. Manual renewal creates new validity + payment record.

4.9 Classes

Aspect	Definition
Physical mapping	ERP Class Type, Class Schedule, Class Session
Description	Class categories, recurring/one-off schedule templates and dated sessions with trainer and capacity.
Purpose	Accurate scheduling and capacity control across any wellness vertical.
Relationships	Class Type 1:N Session; Schedule 1:N Session; Staff 0..N Session; Session 1:N Booking/Attendance.
Business rules	Capacity from configuration (default 30; override per class/session), never hard-coded. Reducing capacity below confirmed bookings is blocked. Trainer assignable per session. Used class types archived not deleted.

4.10 Bookings

Aspect	Definition
Physical mapping	ERP Booking
Description	A reservation of a seat (Booked), a waitlist entry (Waitlisted) or cancellation.
Purpose	Prevents overbooking; increases utilisation with waitlist promotion.
Relationships	Member 1:N Booking; Session 1:N Booking; Membership 1:N Booking.
Business rules	Final validation at commit in one ERP transaction: eligibility, limit, duplicate, capacity (BR-BKG-01). Exactly one confirmation for concurrent last-seat requests. Cancellation cutoff default 4 h (configurable); released seat promotes next eligible waitlisted member. Events emitted only after commit.

4.11 Attendance

Aspect	Definition
Physical mapping	ERP Attendance Record, Attendance Correction
Description	Per-session presence of a member captured by QR, trainer, front desk or admin correction.
Purpose	Trusted engagement data and dispute resolution.
Relationships	Member 1:N Attendance; Session or Event 1:N Attendance; Attendance 1:N Correction.
Business rules	Per session, not per day. One record per member per session. Trainer marks only assigned sessions. Corrections store actor, time, previous, new, reason — immutable and visible in member history. Drop-in behaviour configurable (open decision).

4.12 Events

Aspect	Definition
Physical mapping	ERP Event, Event Registration; config.event_categories
Description	Workshops and studio events with capacity, window, audience and price, plus registrations.
Purpose	New revenue stream and lead source.
Relationships	Event 1:N Registration; Registration N:1 Member XOR Lead; Registration 0..N Payment.
Business rules	Categories configurable. Audience public/members/prospects; members-only hidden from non-members. Paid registration confirmed only after verified payment; unpaid hold releases seat after hold time. Seat limit always enforced.

4.13 Payments

Aspect	Definition
Physical mapping	ERP Payment Record; ops.webhook_inbox, ops.idempotency_keys
Description	Internal payment record (gateway or manual) with provider reference and status Created/Pending/Paid/Failed/Refunded.
Purpose	Revenue integrity and clean reconciliation.

Aspect	Definition
Relationships	Payment N:1 Member/Lead; N:0..1 Membership or Event Registration.
Business rules	Only verified server-side confirmation marks Paid (BR-PAY-02). Idempotent callbacks; delayed callback leaves Pending. Manual entries need actor + reference and are audited. Refunds/adjustments need permission and audit (P2). Currency INR default.

4.14 Staff

Aspect	Definition
Physical mapping	ERP Wellness Staff; iam identity + assignments
Description	Trainer/staff business record (ERP) linked to login identity and role assignments (App DB).
Purpose	Trainer assignment, scope and deactivation with history preserved.
Relationships	Staff 0..1:1 User; Staff N:M Class Type; Staff 1:N Class Session.
Business rules	Create/deactivate with role and studio/branch scope. Deactivated staff cannot log in; historical attribution preserved. Trainers: assigned sessions only, no financial data.

4.15 Notifications

Aspect	Definition
Physical mapping	config.notification_templates/rules; notify.*; ops.outbox_events
Description	Templates, timing rules, per-recipient messages, wire records, delivery status, consents, alerts.
Purpose	Configurable WhatsApp workflows with visible delivery and failure status.
Relationships	Outbox Event 1:N Notification; Notification 1:N WhatsApp Message 1:N Delivery Event.
Business rules	Sent only after commit; failure never reverses a transaction; at most one message per event+recipient (dedupe_key); approved templates only; marketing needs opt-in; mock mode stores would-send records; failed messages surface to admins.

4.16 Reports

Aspect	Definition
Physical mapping	reporting.report_definitions, kpi_snapshots, report_exports
Description	Governed definitions of KPIs/reports and time-bucketed aggregates from ERP.
Purpose	One reporting service feeds dashboards and the AI Assistant so figures match.
Relationships	Definition 1:N Snapshot; Definition 1:N Export (P2).
Business rules	Same metric + period = identical figures in dashboard and Assistant (B2B-RPT-03). Access respects role scope. Snapshots are advisory caches, never used for availability or eligibility.

5. Physical Database Design

Part A defines the PostgreSQL application database (71 tables). Part B defines the ERP DocTypes (20) that hold business records of truth. Column tables list: name, data type, nullability (N = NOT NULL, Y = nullable), default, and constraint/foreign-key/notes. A line under each table lists composite primary keys, unique constraints, indexes and partitioning where not obvious from the columns.

5.0 Global conventions

Topic	Convention
Primary keys	uuid, DEFAULT gen_random_uuid() (UUIDv7 generator when PG 18 available). Composite PKs only for pure link tables and partitioned tables (id, time column).
Tenant column	tenant_id uuid NOT NULL REFERENCES platform.tenants(id) on every tenant-scoped table; indexed as the leading column of every composite index used by tenant queries.
Scope columns	studio_id / branch_id nullable where a record may be scoped narrower than tenant.
Timestamps	created_at, updated_at timestamptz NOT NULL DEFAULT now(); updated_at maintained by a shared trigger platform.set_updated_at(). Append-only tables have created_at only.
Enumerations	varchar + CHECK constraint (not PG ENUM) so values can be extended by expand-migrate-contract without table rewrites.
Text keys	citext for email; E.164 varchar(16) for phone; lower-case slugs for codes.
Soft state	status/is_active/archived columns instead of DELETE for configuration and identity data (BR-GEN-03).
Extensions	pgcrypto (gen_random_uuid, digest), citext, pg_trgm (fuzzy search), btree_gin. pgvector optional and deferred.
Naming	snake_case; tables plural except link/ledger tables; FK constraint names fk_<table>_<col>; index names ix_<table>_<cols>; unique uq_<table>_<cols>.
Foreign keys to ERP	Never a DB FK. Stored as *_erp_ref varchar(140) or erp_docname; integrity verified by adapter + nightly reconciliation.

Part A — PostgreSQL application database

5A · Schema platform — tenancy and entitlements

platform.subscription_tiers [P0 · Configuration]

Platform SaaS tiers (Basic, Premium, Ultra). Separate from studio member plans (BR-MSP-01).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
code	varchar(20)	N		UNIQUE; CHECK (code IN ('basic','premium','ultra'))
name	varchar(60)	N		
description	text	Y		
sort_order	smallint	N	0	
is_active	boolean	N	true	
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

platform.tier_features [P0 · Configuration]

Feature entitlements and limits per tier (AI, WhatsApp automation, studio/branch limits). Feeds BR-AI-11 gating.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tier_id	uuid	N		FK → platform.subscription_tiers(id) ON DELETE CASCADE
feature_key	varchar(80)	N		e.g. ai.admin_assistant, automation.whatsapp, limits.max_branches
enabled	boolean	N	true	
limit_value	integer	Y		NULL = unlimited
config	jsonb	N	'{}'	extra limits (token quota, rate limits)
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

UNIQUE (tier_id, feature_key)

platform.tenants [P0 · Core]

Tenant master: one row per subscribing studio business. Root of every isolation boundary.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
code	varchar(40)	N		UNIQUE; subdomain-safe slug; CHECK (code ~ '^[a-z0-9][a-z0-9-]{1,38}$')
legal_name	varchar(200)	N		
display_name	varchar(120)	N		

Column	Data type	Null	Default	Constraint / FK / notes
tier_id	uuid	N		FK → platform.subscription_tiers(id)
status	varchar(20)	N	'active'	CHECK (status IN ('onboarding','active','suspended','closed'))
timezone	varchar(60)	N	'Asia/Kolkata'	IANA tz
currency	char(3)	N	'INR'	ISO-4217 (INR only in Phase 1, A-07)
locale	varchar(10)	N	'en-IN'	i18n ready (NFR-USB-03)
erp_company	varchar(140)	Y		ERPNext Company name bound to this tenant
data_region	varchar(20)	N	'in-west'	future residency routing
closed_at	timestamptz	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

Indexes: UNIQUE(code); INDEX(status). tenants is NOT tenant-scoped by RLS (platform table); readable only via platform-admin role and tenant resolver function.

platform.tenant_domains [P0 · Configuration]

Host → tenant resolution (AD11). Subdomain in Phase 1, custom domains later.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
host	varchar(253)	N		UNIQUE (global); lower-case; e.g. neha.wellnessos.app
kind	varchar(12)	N	'subdomain'	CHECK (kind IN ('subdomain','custom'))
is_primary	boolean	N	true	
verified_at	timestamptz	Y		custom domain verification (P2)
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

Partial UNIQUE (tenant_id) WHERE is_primary.

platform.tenant_feature_flags [P0 · Configuration]

Per-tenant overrides on top of tier entitlements. Resolution: platform default → tier → tenant override (§6.3 of architecture).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
feature_key	varchar(80)	N		
enabled	boolean	N		
config	jsonb	N	'{}'	
reason	text	Y		why overridden
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger
created_by	uuid	Y		actor user_id or service principal id

UNIQUE (tenant_id, feature_key).

platform.studios [P0 · Core]

A studio under a tenant (Tenant → Studio → Branch hierarchy).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
code	varchar(40)	N		UNIQUE (tenant_id, code)
name	varchar(160)	N		
address	jsonb	Y		structured postal address
phone	varchar(20)	Y		E.164
email	varchar(254)	Y		
status	varchar(20)	N	'active'	CHECK (status IN ('active','inactive','archived'))
erp_ref	varchar(140)	Y		ERP Studio doc name (if mirrored)
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

platform.branches [P0 · Core]

Branch (location) within a studio; branch-ready from day one (NFR-SCL-01).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
studio_id	uuid	N		FK → platform.studios(id)
code	varchar(40)	N		UNIQUE (tenant_id, studio_id, code)
name	varchar(160)	N		
address	jsonb	Y		
timezone	varchar(60)	Y		overrides tenant timezone
status	varchar(20)	N	'active'	CHECK (status IN ('active','inactive','archived'))
erp_ref	varchar(140)	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

Composite FK (tenant_id, studio_id) → studios(tenant_id, id) prevents cross-tenant parent linkage (requires UNIQUE (tenant_id,id) on studios).

platform.tenant_subscriptions [P2 · Configuration]

SaaS billing state of a tenant (self-service billing, Phase 2).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
tier_id	uuid	N		FK → platform.subscription_tiers(id)
status	varchar(20)	N	'active'	CHECK (status IN ('trial','active','past_due','cancelled'))
period_start	date	N		
period_end	date	Y		
provider_ref	varchar(120)	Y		billing provider subscription id
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

platform.tenant_erp_routes [P1 · Configuration]

Tenant → ERP site/company routing map used by the ERP adapter; enables site-per-tenant scaling without caller change (§6.4).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
erp_site	varchar(200)	N		Frappe site/base URL key (not a secret)
erp_company	varchar(140)	N		
custom_app_version	varchar(20)	Y		last verified wellness_studio version
credential_ref	varchar(120)	N		pointer to secret-store entry (never the secret)
is_active	boolean	N	true	
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

UNIQUE (tenant_id) WHERE is_active.

5A · Schema iam — identity and access

iam.users [P0 · Core]

Application identities: staff (email+password) and members (phone OTP / Google). Members are NOT ERP users; linkage via iam.user_erp_links.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
user_type	varchar(10)	N		CHECK (user_type IN ('staff','member','platform'))
email	citext	Y		lower-cased; UNIQUE (tenant_id,email) WHERE email IS NOT NULL
phone_e164	varchar(16)	Y		UNIQUE (tenant_id,phone_e164) WHERE NOT NULL; CHECK (phone_e164 ~ '^\+[1-9][0-9]{7,14}$')
display_name	varchar(160)	N		
status	varchar(20)	N	'active'	CHECK (status IN ('invited','active','locked','deactivated'))
must_reset_password	boolean	N	false	mandatory reset (NFR-SEC-02)
last_login_at	timestamptz	Y		
deactivated_at	timestamptz	Y		history retained (B2B-STF-01)
created_at	timestamptz	N	now()	

Column	Data type	Null	Default	Constraint / FK / notes
updated_at	timestamptz	N	now()	set by trigger

platform users have tenant_id = the reserved platform tenant row. At least one of email/phone must be present: CHECK (email IS NOT NULL OR phone_e164 IS NOT NULL).

iam.user_credentials [P0 · Core]

Password hash for staff. Kept apart from users so it can carry stricter grants and never appear in generic selects.

Column	Data type	Null	Default	Constraint / FK / notes
user_id	uuid	N		PK, FK → iam.users(id) ON DELETE CASCADE
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
password_hash	text	N		argon2id encoded string
password_changed_at	timestamptz	N	now()	
failed_attempts	smallint	N	0	lockout counter
locked_until	timestamptz	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

iam.auth_identities [P0 · Core]

External/alternative login identities (verified phone, Google subject). Account linking resolves on verified phone to avoid duplicates (B2C-PRF-01).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
user_id	uuid	N		FK → iam.users(id) ON DELETE CASCADE
provider	varchar(20)	N		CHECK (provider IN ('phone_otp','google','password'))
provider_subject	varchar(255)	N		phone E.164 or Google sub
verified_at	timestamptz	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

UNIQUE (tenant_id, provider, provider_subject).

iam.roles [P0 · Configuration]

Roles as permission bundles: owner, admin, front_desk, trainer, member, visitor, platform_admin. System roles are seeded and immutable; tenants may add custom roles later.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	Y		NULL = system role; FK → platform.tenants(id)
code	varchar(40)	N		UNIQUE (COALESCE(tenant_id, zero-uuid), code)
name	varchar(80)	N		
is_system	boolean	N	false	
description	text	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

iam.permissions [P0 · Configuration]

Global catalogue of granular permissions (resource:action), e.g. members:create, bookings:on_behalf, payments:record. Platform-defined, code-seeded.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
code	varchar(80)	N		UNIQUE; pattern '<resource>:<action>'
resource	varchar(40)	N		
action	varchar(40)	N		
risk_class	varchar(10)	N	'read'	CHECK (risk_class IN ('read','write','sensitive')) — used by AI tool registry
description	text	Y		

iam.role_permissions [P0 · Configuration]

Many-to-many: permissions bundled into a role.

Column	Data type	Null	Default	Constraint / FK / notes
role_id	uuid	N		FK → iam.roles(id) ON DELETE CASCADE
permission_id	uuid	N		FK → iam.permissions(id) ON DELETE CASCADE
created_at	timestamptz	N	now()	append-only

PK (role_id, permission_id).

iam.user_role_assignments [P0 · Core]

Assigns a role to a user with tenant/studio/branch scope (evaluation chain: identity → role → permission → scope → ownership).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
user_id	uuid	N		FK → iam.users(id)
role_id	uuid	N		FK → iam.roles(id)
studio_id	uuid	Y		FK → platform.studios(id); NULL = all studios of tenant (Owner)
branch_id	uuid	Y		FK → platform.branches(id); NULL = all branches of studio
valid_from	timestamptz	N	now()	
valid_to	timestamptz	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger
created_by	uuid	Y		actor user_id or service principal id

UNIQUE (tenant_id,user_id,role_id,COALESCE(studio_id,zero),COALESCE(branch_id,zero)). Role changes effective next request (no cached permission beyond request).

iam.permission_grants [P0 · Core]

Explicit, scoped, revocable special grants (e.g. trainer-assisted booking) — B2B-STF-02.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
user_id	uuid	N		FK → iam.users(id)
permission_id	uuid	N		FK → iam.permissions(id)
studio_id	uuid	Y		FK → platform.studios(id)
branch_id	uuid	Y		FK → platform.branches(id)
granted_by	uuid	N		FK → iam.users(id)
granted_at	timestamptz	N	now()	
revoked_at	timestamptz	Y		soft revoke; row retained for audit
revoked_by	uuid	Y		FK → iam.users(id)
reason	text	Y		

Partial UNIQUE (tenant_id,user_id,permission_id,COALESCE(studio_id,zero),COALESCE(branch_id,zero)) WHERE revoked_at IS NULL.

iam.user_erp_links [P0 · Core]

Identity mapping user ↔ ERP Member/Staff document (reference by ID only; no business fields copied — architecture §7.3).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
user_id	uuid	N		FK → iam.users(id) ON DELETE CASCADE
erp_doctype	varchar(40)	N		CHECK (erp_doctype IN ('Member','Wellness Staff'))
erp_docname	varchar(140)	N		e.g. MEM-00042
linked_at	timestamptz	N	now()	
link_method	varchar(20)	N	'verified_phone'	CHECK (link_method IN ('verified_phone','admin','onboarding'))

UNIQUE (tenant_id,erp_doctype,erp_docname); UNIQUE (tenant_id,user_id,erp_doctype).

iam.auth_sessions [P0 · Core]

Refresh-token sessions (rotation). Access tokens are stateless JWTs bound to tenant claim.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
user_id	uuid	N		FK → iam.users(id) ON DELETE CASCADE
refresh_token_hash	char(64)	N		SHA-256; UNIQUE
family_id	uuid	N		rotation family; reuse of old token revokes family
ip_address	inet	Y		
user_agent	text	Y		
expires_at	timestamptz	N		
revoked_at	timestamptz	Y		
created_at	timestamptz	N	now()	

Column	Data type	Null	Default	Constraint / FK / notes
updated_at	timestamptz	N	now()	set by trigger

iam.otp_challenges [P0 · Core]

Phone OTP challenges for members (provider TBD, S7). Rate-limited and short-lived.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
phone_e164	varchar(16)	N		
code_hash	char(64)	N		never store plain OTP
purpose	varchar(20)	N	'login'	CHECK (purpose IN ('login','link','phone_change'))
attempts	smallint	N	0	CHECK (attempts <= 5)
expires_at	timestamptz	N		
consumed_at	timestamptz	Y		
created_at	timestamptz	N	now()	append-only

Cleanup: delete rows 24h after expires_at.

iam.mfa_factors [P2 · Core]

TOTP factors for Owner/Admin (MFA mandatory before production; enforcement Phase 2).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
user_id	uuid	N		FK → iam.users(id) ON DELETE CASCADE
factor_type	varchar(10)	N	'totp'	CHECK (factor_type IN ('totp'))
secret_enc	bytea	N		envelope-encrypted secret
verified_at	timestamptz	Y		
recovery_codes_hash	jsonb	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

iam.service_principals [P1 · Core]

Non-interactive identities: n8n, ERP callbacks, and future agents. Scoped, rotated credentials.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	Y		NULL = platform-wide (e.g. ERP callback); FK → platform.tenants(id)
kind	varchar(20)	N		CHECK (kind IN ('n8n','erp_callback','agent','worker'))
name	varchar(80)	N		UNIQUE (COALESCE(tenant_id,zero), name)
key_id	varchar(40)	N		UNIQUE; public key/HMAC key identifier
secret_ref	varchar(120)	N		secret-store pointer
allowed_scopes	text[]	N	'{}'	permission codes ceiling
is_active	boolean	N	true	
rotated_at	timestamptz	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

5A · Schema config — tenant configuration

config.tenant_branding [P0 · Configuration]

Studio profile & branding tokens injected into portals (B2B-SET-01).

Column	Data type	Null	Default	Constraint / FK / notes
tenant_id	uuid	N		PK, FK → platform.tenants(id)
studio_name	varchar(160)	N		
logo_url	text	Y		object-storage path (tenant-prefixed)
primary_color	char(7)	N	'#1F3A5F'	CHECK (primary_color ~ '^#[0-9A-Fa-f]{6}$')
secondary_color	char(7)	Y		
contact_email	varchar(254)	Y		
contact_phone	varchar(20)	Y		
theme_tokens	jsonb	N	'{}'	extra design tokens
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger
created_by	uuid	Y		actor user_id or service principal id

config.tenant_settings [P0 · Configuration]

Typed key/value operational settings (default capacity 30, cancellation cutoff 4h, hold minutes, drop-in policy, reminder toggles). Never hard-coded (BR-GEN-05).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
studio_id	uuid	Y		FK → platform.studios(id); NULL = tenant-wide
setting_key	varchar(80)	N		e.g. booking.default_capacity
value_json	jsonb	N		validated against registry schema
value_type	varchar(12)	N		CHECK (value_type IN ('int','bool','string','json','duration'))
version	integer	N	1	optimistic concurrency
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger
created_by	uuid	Y		actor user_id or service principal id

UNIQUE (tenant_id, COALESCE(studio_id,zero), setting_key). Seed defaults: booking.default_capacity=30, booking.cancel_cutoff_minutes=240, event.payment_hold_minutes=15, attendance.allow_dropin=false.

config.event_categories [P0 · Configuration]

Configurable event categories (Workshop, Internal Studio Event, Public Event, Special Session — BR-EVT-01). ERP Event stores category code.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
code	varchar(40)	N		UNIQUE (tenant_id, code)
name	varchar(80)	N		
default_audience	varchar(12)	N	'public'	CHECK (default_audience IN ('public','members','prospects'))
reminder_profile	varchar(20)	N	'event'	selects reminder rule set (event vs workshop)
is_active	boolean	N	true	archive, never delete
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

config.notification_templates [P0 · Configuration]

Tenant template registry: logical template with channel and approved WhatsApp template name (BR-NTF-05).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
code	varchar(60)	N		e.g. booking_confirmation
channel	varchar(10)	N	'whatsapp'	CHECK (channel IN ('whatsapp','email','in_app'))
category	varchar(15)	N	'transactional'	CHECK (category IN ('transactional','marketing'))
provider_template_name	varchar(120)	Y		Meta-approved name
language	varchar(10)	N	'en'	
body	text	N		text with {{variables}}
variables	jsonb	N	'[]'	declared variable list
approval_status	varchar(15)	N	'draft'	CHECK (approval_status IN ('draft','pending','approved','rejected'))
is_active	boolean	N	true	
version	integer	N	1	
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger
created_by	uuid	Y		actor user_id or service principal id

UNIQUE (tenant_id, code, channel, language).

config.notification_rules [P0 · Configuration]

Trigger mapping and timing: event type → template, offset, enablement (B2B-SET-02). n8n reads this; timings never live in workflow JSON.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
event_type	varchar(60)	N		e.g. booking.confirmed, class.reminder, membership.expiring
template_id	uuid	N		FK → config.notification_templates(id)
offset_minutes	integer	N	0	negative = before, positive = after, 0 = immediate
anchor	varchar(20)	N	'event'	CHECK (anchor IN ('event','session_start','expiry','registration'))

Column	Data type	Null	Default	Constraint / FK / notes
is_enabled	boolean	N	true	
sequence_no	smallint	N	1	orders multiple reminders (e.g. 12h, 1h)
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger
created_by	uuid	Y		actor user_id or service principal id

UNIQUE (tenant_id,event_type,sequence_no).

config.faq_entries [P1 · Configuration]

Tenant-approved knowledge used by Enquiry Assistant (BR-AI-08). Only approved rows are retrievable.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
question	text	N		
answer	text	N		
tags	text[]	N	'{}'	
status	varchar(12)	N	'draft'	CHECK (status IN ('draft','approved','archived'))
approved_by	uuid	Y		FK → iam.users(id)
approved_at	timestamptz	Y		
search_tsv	tsvector	N		GENERATED ALWAYS AS (to_tsvector('english', question
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

GIN(search_tsv). Optional embedding column (vector) is deferred until retrieval evaluation demands it.

5A · Schema ops — reliability, webhooks, reconciliation

ops.outbox_events [P0 · Transactional]

Transactional outbox (AD06): events inserted in the same transaction as the audit record, after the ERP call succeeded. Dispatcher delivers to n8n with retry and dedup.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
event_type	varchar(80)	N		e.g. booking.confirmed
subject_type	varchar(40)	N		ERP doctype or app table
subject_ref	varchar(140)	N		ERP docname or uuid text
payload	jsonb	N		minimal envelope data; no PII beyond ids
correlation_id	uuid	N		
occurred_at	timestamptz	N	now()	
status	varchar(12)	N	'pending'	CHECK (status IN ('pending','dispatching','delivered','failed','dead'))
attempts	smallint	N	0	
next_attempt_at	timestamptz	N	now()	back-off schedule
last_error	text	Y		
delivered_at	timestamptz	Y		
created_at	timestamptz	N	now()	append-only

Envelope for consumers: (id, event_type, tenant_id, occurred_at, subject, correlation_id). Consumers idempotent on id. Partial INDEX (next_attempt_at) WHERE status IN (pending,failed). Partition monthly (P1).

ops.idempotency_keys [P0 · Transactional]

Request-level idempotency for public/payment/AI endpoints and callbacks (NFR-REL-01).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
scope	varchar(40)	N		endpoint or operation name
key	varchar(200)	N		client or derived key
request_hash	char(64)	N		detects key reuse with different payload
response_status	smallint	Y		
response_body	jsonb	Y		
expires_at	timestamptz	N	now() + interval '48 hours'	
created_at	timestamptz	N	now()	append-only

UNIQUE (tenant_id, scope, key).

ops.webhook_inbox [P0 · Transactional]

Raw inbound webhook receipts (Razorpay, WhatsApp status, ERP callbacks, n8n) with signature result; provides replay and dedupe.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	Y		resolved after verification; FK → platform.tenants(id)
provider	varchar(20)	N		CHECK (provider IN ('razorpay','whatsapp','erp','n8n','email'))
provider_event_id	varchar(200)	N		provider event id
signature_valid	boolean	N		
payload	jsonb	N		
headers	jsonb	N	'{}'	redacted
status	varchar(12)	N	'received'	CHECK (status IN ('received','processed','ignored','failed'))
received_at	timestamptz	N	now()	
processed_at	timestamptz	Y		
error	text	Y		

UNIQUE (provider, provider_event_id). Payment state changes only from rows with signature_valid = true (NFR-SEC-07).

ops.reconciliation_runs [P1 · Transactional]

Nightly reconciliation job header: orphan ERP references, outbox events without outcome, stale Pending payments (§7.3).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	Y		NULL = all tenants
run_type	varchar(30)	N		CHECK (run_type IN ('erp_orphans','outbox_outcome','payment_pending','notification_gap'))
started_at	timestamptz	N	now()	
finished_at	timestamptz	Y		
status	varchar(12)	N	'running'	CHECK (status IN ('running','ok','findings','failed'))
summary	jsonb	N	'{}'	

ops.reconciliation_findings [P1 · Transactional]

Individual discrepancies reported to admins.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
run_id	uuid	N		FK → ops.reconciliation_runs(id) ON DELETE CASCADE
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
finding_type	varchar(40)	N		
subject_ref	varchar(140)	N		
detail	jsonb	N	'{}'	
resolution_status	varchar(12)	N	'open'	CHECK (resolution_status IN ('open','acknowledged','resolved'))
resolved_by	uuid	Y		FK → iam.users(id)
resolved_at	timestamptz	Y		
created_at	timestamptz	N	now()	append-only

ops.erp_contract_versions [P2 · Configuration]

Adapter ↔ custom-app compatibility log written at startup version check.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
adapter_version	varchar(20)	N		
erp_app_version	varchar(20)	N		
compatible	boolean	N		
checked_at	timestamptz	N	now()	
detail	jsonb	N	'{}'	

5A · Schema reporting — report registry and KPI snapshots

reporting.report_definitions [P1 · Configuration]

Registry of governed report/KPI queries shared by dashboards and the Assistant (B2B-RPT-03). Definition, not results.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	Y		NULL = platform-standard definition
code	varchar(60)	N		e.g. memberships_expiring, low_attendance_classes
name	varchar(120)	N		

Column	Data type	Null	Default	Constraint / FK / notes
source	varchar(10)	N	'erp'	CHECK (source IN ('erp','app_db','hybrid'))
query_spec	jsonb	N		parameter schema, ERP method or SQL view name
required_permission	varchar(80)	N		FK-by-code → iam.permissions(code)
is_active	boolean	N	true	
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

UNIQUE (COALESCE(tenant_id,zero), code).

reporting.kpi_snapshots [P1 · Transactional]

Time-bucketed KPI aggregates pulled from ERP (scheduled) to keep dashboards under 3 s (NFR-PRF-02). Never used for availability/eligibility.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
studio_id	uuid	Y		FK → platform.studios(id)
kpi_code	varchar(60)	N		active_members, monthly_revenue, occupancy...
period_kind	varchar(8)	N		CHECK (period_kind IN ('hour','day','week','month'))
period_start	timestamptz	N		
value_numeric	numeric(18,4)	Y		
dimensions	jsonb	N	'{}'	plan, class_type...
computed_at	timestamptz	N	now()	
source_run_id	uuid	Y		

UNIQUE (tenant_id,COALESCE(studio_id,zero),kpi_code,period_kind,period_start,dimensions). Partition by period_start (yearly).

reporting.report_exports [P2 · Transactional]

CSV/PDF export jobs (Phase 2); same filters and permissions as screen.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
requested_by	uuid	N		FK → iam.users(id)
report_definition_id	uuid	N		FK → reporting.report_definitions(id)
filters	jsonb	N	'{}'	
format	varchar(4)	N		CHECK (format IN ('csv','pdf'))
status	varchar(12)	N	'queued'	CHECK (status IN ('queued','running','ready','failed','expired'))
file_path	text	Y		tenant-prefixed object path
expires_at	timestamptz	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

5A · Schema notify — notifications and automation

notify.notification_messages [P0 · Transactional]

Logical notification per (event, recipient, template): the Notification Log shown to admins (B2B-NTF-01). Idempotency anchor (BR-NTF-03).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
outbox_event_id	uuid	Y		FK → ops.outbox_events(id)
event_type	varchar(80)	N		
template_id	uuid	Y		FK → config.notification_templates(id)
channel	varchar(10)	N	'whatsapp'	CHECK (channel IN ('whatsapp','email','in_app'))
recipient_kind	varchar(10)	N		CHECK (recipient_kind IN ('member','lead','staff','registrant'))
recipient_erp_ref	varchar(140)	Y		ERP Member/Lead docname (no copy of profile)
recipient_address_hash	char(64)	Y		SHA-256 of phone/email for dedupe without storing PII here
recipient_address_enc	bytea	Y		envelope-encrypted destination, needed only to send/retry
dedupe_key	varchar(300)	N		event_id + recipient + template
mode	varchar(6)	N	'mock'	CHECK (mode IN ('mock','live'))
status	varchar(12)	N	'queued'	CHECK (status IN ('queued','skipped_consent','sent','delivered','read','failed','mock_recorded'))
attempts	smallint	N	0	

Column	Data type	Null	Default	Constraint / FK / notes
failure_reason	text	Y		
correlation_id	uuid	N		
scheduled_for	timestamptz	Y		
sent_at	timestamptz	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

UNIQUE (tenant_id, dedupe_key). INDEX (tenant_id, status, created_at DESC). Partition monthly. Failed messages never reverse the business transaction.

notify.whatsapp_messages [P0 · Transactional]

Provider-level WhatsApp wire record (live send or would-send mock record — BR-NTF-04).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
notification_id	uuid	N		FK → notify.notification_messages(id)
direction	varchar(8)	N	'outbound'	CHECK (direction IN ('outbound','inbound'))
provider_message_id	varchar(120)	Y		Meta wamid; UNIQUE WHERE NOT NULL
template_name	varchar(120)	N		
template_language	varchar(10)	N	'en'	
to_hash	char(64)	N		recipient hash
payload_redacted	jsonb	N	'{}'	variables with PII masked
status	varchar(12)	N	'queued'	CHECK (status IN ('queued','mock','accepted','sent','delivered','read','failed','template_rejected'))
error_code	varchar(40)	Y		
mock	boolean	N	true	
sent_at	timestamptz	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

notify.notification_delivery_events [P0 · Transactional]

Status callbacks (sent/delivered/read/failed) appended per message; source for delivery success KPI.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
whatsapp_message_id	uuid	N		FK → notify.whatsapp_messages(id)
status	varchar(12)	N		
provider_timestamp	timestamptz	Y		
raw	jsonb	N	'{}'	
webhook_inbox_id	uuid	Y		FK → ops.webhook_inbox(id)
created_at	timestamptz	N	now()	append-only

notify.email_logs [P2 · Transactional]

Transactional email send log (Phase 2). Same lifecycle as WhatsApp records; provider TBD.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
notification_id	uuid	N		FK → notify.notification_messages(id)
provider	varchar(30)	N		
provider_message_id	varchar(160)	Y		
subject	varchar(300)	N		
to_hash	char(64)	N		
status	varchar(12)	N	'queued'	CHECK (status IN ('queued','sent','delivered','bounced','complained','failed'))
error	text	Y		
sent_at	timestamptz	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

notify.communication_consents [P1 · Transactional]

Opt-in/opt-out per person, channel and category; enforced by API before releasing a send (NFR-CMP-02).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
subject_kind	varchar(10)	N		CHECK (subject_kind IN ('member','lead','registrant'))
subject_erp_ref	varchar(140)	N		
channel	varchar(10)	N		
category	varchar(15)	N		CHECK (category IN ('transactional','marketing'))
status	varchar(10)	N		CHECK (status IN ('opted_in','opted_out'))
source	varchar(30)	N		form, whatsapp_reply, admin, import
evidence	jsonb	N	'{}'	consent text version, IP, timestamp
recorded_at	timestamptz	N	now()	

Append-only history; effective state = latest row per (tenant, subject, channel, category). INDEX (tenant_id, subject_erp_ref, channel, category, recorded_at DESC).

notify.suppression_list [P2 · Transactional]

Hard-bounce/complaint/opt-out suppression (email, Phase 2).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
channel	varchar(10)	N		
address_hash	char(64)	N		
reason	varchar(20)	N		CHECK (reason IN ('bounce','complaint','opt_out','manual'))
created_at	timestamptz	N	now()	

UNIQUE (tenant_id, channel, address_hash).

notify.workflow_executions [P0 · Transactional]

One row per n8n workflow run reported back to the API (orchestration trace; n8n is never a source of truth — AD07).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
workflow_code	varchar(60)	N		e.g. booking-confirmation, send-message
n8n_execution_id	varchar(80)	Y		
outbox_event_id	uuid	Y		FK → ops.outbox_events(id)
trigger_kind	varchar(12)	N		CHECK (trigger_kind IN ('event','schedule','manual'))
status	varchar(12)	N	'running'	CHECK (status IN ('running','succeeded','failed','retrying','dead'))
attempt	smallint	N	1	
started_at	timestamptz	N	now()	
finished_at	timestamptz	Y		
error	text	Y		
correlation_id	uuid	N		

Partition monthly. INDEX (tenant_id, status, started_at DESC).

notify.automation_audit_trails [P1 · Audit]

Append-only trail of automation decisions: consent check, dedupe skip, mode switch, retry, alert raised.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
workflow_execution_id	uuid	Y		FK → notify.workflow_executions(id)
notification_id	uuid	Y		FK → notify.notification_messages(id)
step	varchar(40)	N		render_template, consent_check, dedupe, send, status_update, alert
decision	varchar(20)	N		proceeded, skipped, retried, failed
detail	jsonb	N	'{}'	
correlation_id	uuid	N		
created_at	timestamptz	N	now()	append-only

notify.admin_alerts [P0 · Transactional]

Failure visibility (100% of failed runs surfaced — Automation KPI): admin tasks/alerts for persistent failures.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
studio_id	uuid	Y		FK → platform.studios(id)
alert_type	varchar(40)	N		message_failed, template_rejected, outbox_dead, payment_mismatch
severity	varchar(8)	N	'warning'	CHECK (severity IN ('info','warning','critical'))
subject_ref	varchar(140)	Y		
message	text	N		
status	varchar(12)	N	'open'	CHECK (status IN ('open','acknowledged','resolved'))
acknowledged_by	uuid	Y		FK → iam.users(id)
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

5A · Schema ai — AI data

ai.ai_sessions [P0 · AI]

Envelope for an assistant chat session: principal, assistant type, surface, expiry. Conversation context is temporary (BR-AI-07).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
assistant_type	varchar(20)	N		CHECK (assistant_type IN ('admin','enquiry','booking','recommendation','feedback'))
principal_kind	varchar(10)	N		CHECK (principal_kind IN ('staff','member','visitor'))
user_id	uuid	Y		FK → iam.users(id); NULL for visitor
visitor_token_hash	char(64)	Y		anonymous enquiry sessions
role_snapshot	text[]	N	'{}'	permissions resolved at session start; re-checked per tool call
status	varchar(10)	N	'active'	CHECK (status IN ('active','closed','expired'))
correlation_id	uuid	N		
started_at	timestamptz	N	now()	
last_activity_at	timestamptz	N	now()	
expires_at	timestamptz	N		

INDEX (tenant_id, user_id, started_at DESC).

ai.ai_conversations [P0 · AI]

Message-level conversation store (turns). PII-minimised before storage; TTL purge; never becomes business data except via a governed service.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
session_id	uuid	N		FK → ai.ai_sessions(id) ON DELETE CASCADE
turn_no	integer	N		
role	varchar(10)	N		CHECK (role IN ('user','assistant','tool','system'))
content_redacted	text	N		PII-minimised text
tool_call_id	uuid	Y		FK → ai.ai_tool_call_audit(id)
token_count	integer	Y		
created_at	timestamptz	N	now()	
purge_after	timestamptz	N		created_at + retention window

UNIQUE (session_id, turn_no). Partial INDEX (purge_after).

ai.ai_pending_actions [P0 · AI]

Two-phase confirmation store (AD09): proposed write actions with FIXED arguments; only stored arguments execute after confirmation.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
session_id	uuid	N		FK → ai.ai_sessions(id)
requested_by	uuid	N		FK → iam.users(id)
tool_name	varchar(80)	N		e.g. createMember
arguments	jsonb	N		frozen at proposal time
arguments_hash	char(64)	N		SHA-256; verified before execution

Column	Data type	Null	Default	Constraint / FK / notes
confirmation_summary	text	N		text shown to user (relative dates resolved)
status	varchar(12)	N	'proposed'	CHECK (status IN ('proposed','confirmed','rejected','expired','executed','failed'))
expires_at	timestamptz	N		
decided_at	timestamptz	Y		
result	jsonb	Y		actual service result (record id or precise error)
executed_at	timestamptz	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

ai.ai_tool_call_audit [P0 · Audit]

Immutable AI audit: one record per model-initiated tool call — request, tool, arguments, decision, approval, result (NFR-AUD-03).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
session_id	uuid	N		FK → ai.ai_sessions(id)
pending_action_id	uuid	Y		FK → ai.ai_pending_actions(id)
actor_user_id	uuid	Y		invoking user
assistant_type	varchar(20)	N		
tool_name	varchar(80)	N		
risk_class	varchar(10)	N		read
request_text_redacted	text	Y		
arguments	jsonb	N	'{}'	
permission_decision	varchar(10)	N		CHECK (permission_decision IN ('allowed','denied'))
approval	varchar(12)	N	'not_required'	CHECK (approval IN ('not_required','confirmed','rejected'))
result_status	varchar(10)	N		CHECK (result_status IN ('success','error','denied'))
result	jsonb	Y		bounded, redacted
model_id	varchar(80)	Y		
prompt_version	varchar(40)	Y		
correlation_id	uuid	N		
created_at	timestamptz	N	now()	append-only

Append-only (no UPDATE/DELETE grant). Partition monthly. Also mirrored by a summarising row in audit.audit_logs (actor_type = ai).

ai.ai_recommendations [P1 · AI]

Rules-constrained recommendations produced for a member; stores the candidate set, ranking and acceptance for KPI tracking (B2C-REC-01/02).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
member_erp_ref	varchar(140)	N		ERP Member docname
session_id	uuid	Y		FK → ai.ai_sessions(id)
source	varchar(12)	N	'rules'	CHECK (source IN ('rules','llm_ranked'))
inputs	jsonb	N	'{}'	goal, level, filters
candidates	jsonb	N		ordered [{session_ref, reason, rank}] (all eligible & available at generation time)
generated_at	timestamptz	N	now()	
expires_at	timestamptz	N		
accepted_session_ref	varchar(140)	Y		set when member books a suggested session
accepted_at	timestamptz	Y		

ai.ai_feedback_submissions [P1 · AI]

Raw member post-class comment (the record of truth). Referenced ERP session; raw text visible only to Owner/Admin.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
member_erp_ref	varchar(140)	N		
class_session_ref	varchar(140)	N		ERP Class Session docname
class_type_ref	varchar(140)	Y		
rating	smallint	Y		CHECK (rating BETWEEN 1 AND 5)
comment_text	text	N		
submitted_at	timestamptz	N	now()	

UNIQUE (tenant_id, member_erp_ref, class_session_ref). Feedback is optional; no automatic member action.

ai.ai_feedback_analysis [P1 · AI]

AI classification of a feedback comment: sentiment, topics, theme (advisory only). Stored separately from raw text.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
submission_id	uuid	N		FK → ai.ai_feedback_submissions(id) ON DELETE CASCADE
sentiment	varchar(10)	N		CHECK (sentiment IN ('positive','neutral','negative'))
topics	text[]	N	'{}'	Trainer, Class Content, Music, Timing, Duration
theme	text	Y		one-sentence key point
confidence	numeric(4,3)	Y		
model_id	varchar(80)	N		
prompt_version	varchar(40)	N		
human_label	varchar(10)	Y		reviewer label for ≥80% agreement sampling
analyzed_at	timestamptz	N	now()	

UNIQUE (submission_id, prompt_version).

ai.ai_feedback_theme_summaries [P1 · AI]

Weekly/per-session theme roll-up shown to admins (aggregate only — trainers never see individual comments).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
scope_kind	varchar(12)	N		CHECK (scope_kind IN ('session','class_type','week'))
scope_ref	varchar(140)	N		
period_start	date	Y		
period_end	date	Y		
sentiment_counts	jsonb	N	'{}'	
top_topics	jsonb	N	'[]'	
summary_text	text	Y		
generated_at	timestamptz	N	now()	

ai.ai_usage_logs [P1 · AI]

Per-call LLM usage for cost and quota control per tenant/tier (integration §11).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
session_id	uuid	Y		FK → ai.ai_sessions(id)
assistant_type	varchar(20)	N		
provider	varchar(30)	N		
model_id	varchar(80)	N		
prompt_tokens	integer	N	0	
completion_tokens	integer	N	0	
latency_ms	integer	Y		
cost_micros	bigint	Y		cost in micro-INR/USD
outcome	varchar(10)	N		CHECK (outcome IN ('ok','timeout','error','blocked'))
created_at	timestamptz	N	now()	

Partition monthly. Rollup table ai.ai_usage_daily (tenant, day, tokens, cost) is a P1 materialised view for quota checks.

ai.ai_prompt_versions [P1 · Configuration]

Registry of prompt template versions used (files versioned in repo; DB row records identity/hash for traceability).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
assistant_type	varchar(20)	N		
version	varchar(40)	N		
content_sha256	char(64)	N		
is_active	boolean	N	false	
created_at	timestamptz	N	now()	

UNIQUE (assistant_type, version).

ai.ai_eval_runs [P1 · AI]

Evaluation harness runs (curated query set + Create Member scenarios) with recorded pass rates — supports 85% / 95% / zero-false-success targets.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
suite	varchar(40)	N		admin_queries, create_member, enquiry_grounding...
git_sha	varchar(40)	N		
model_id	varchar(80)	N		
total_cases	integer	N		
passed_cases	integer	N		
false_success_count	integer	N	0	must be 0
started_at	timestamptz	N	now()	
results	jsonb	N	'[]'	per-case outcomes

5A · Schema agent — agent runtime data

agent.agent_registry [P2 · Agent]

Catalogue of agents (CRM, Operations, Marketing, Advertising, Growth, Studio Insights, Session Summary) with domain and phase. Schema reserved from Phase 1 for agent-readiness.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
code	varchar(40)	N		UNIQUE; crm, operations, marketing, advertising, growth, insights, session_summary
name	varchar(80)	N		
domain	varchar(40)	N		single domain toolset
phase	varchar(4)	N		CHECK (phase IN ('P2','P3','F'))
default_approval	varchar(12)	N	'required'	CHECK (default_approval IN ('required','advisory','none'))
is_active	boolean	N	false	
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

agent.agent_tenant_configs [P2 · Agent]

Per-tenant enablement, schedule and service principal for each agent; gated by tier and feature flag (BR-AI-11).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
agent_id	uuid	N		FK → agent.agent_registry(id)
service_principal_id	uuid	N		FK → iam.service_principals(id)
enabled	boolean	N	false	
schedule_cron	varchar(60)	Y		
settings	jsonb	N	'{}'	
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

UNIQUE (tenant_id, agent_id).

agent.agent_tool_grants [P2 · Agent]

Tools an agent may call (only its domain). References the tool registry by name; permission ceiling per grant.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
agent_id	uuid	N		FK → agent.agent_registry(id) ON DELETE CASCADE
tool_name	varchar(80)	N		
permission_code	varchar(80)	N		FK-by-code → iam.permissions(code)
risk_class	varchar(10)	N		read

UNIQUE (agent_id, tool_name).

agent.agent_tasks [P2 · Agent]

Unit of agent work created by trigger (event, schedule, user).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
agent_id	uuid	N		FK → agent.agent_registry(id)
trigger_kind	varchar(10)	N		CHECK (trigger_kind IN ('event','schedule','user'))

Column	Data type	Null	Default	Constraint / FK / notes
trigger_ref	varchar(140)	Y		outbox event id or user id
on_behalf_of	uuid	Y		FK → iam.users(id); permissions ceiling
goal	text	N		
status	varchar(12)	N	'queued'	CHECK (status IN ('queued','running','awaiting_approval','done','failed','cancelled'))
priority	smallint	N	5	
correlation_id	uuid	N		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

agent.agent_executions [P2 · Agent]

One execution attempt of a task (run of the agent loop).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
task_id	uuid	N		FK → agent.agent_tasks(id) ON DELETE CASCADE
attempt	smallint	N	1	
status	varchar(12)	N	'running'	CHECK (status IN ('running','succeeded','failed','timed_out'))
model_id	varchar(80)	Y		
tokens_used	integer	Y		
started_at	timestamptz	N	now()	
finished_at	timestamptz	Y		
error	text	Y		

agent.agent_decisions [P2 · Agent]

Proposed actions / drafts and the human approval decision (approval queue, Phase 2). Mirrors ai_pending_actions semantics.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
task_id	uuid	N		FK → agent.agent_tasks(id)
execution_id	uuid	Y		FK → agent.agent_executions(id)
decision_type	varchar(20)	N		draft_message, schedule_change, recommendation, alert
proposal	jsonb	N		frozen arguments
proposal_hash	char(64)	N		
requires_approval	boolean	N	true	
status	varchar(12)	N	'pending'	CHECK (status IN ('pending','approved','rejected','executed','expired','failed'))
decided_by	uuid	Y		FK → iam.users(id)
decided_at	timestamptz	Y		
decision_note	text	Y		
result	jsonb	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

agent.agent_memory [P2 · Agent]

Scoped working/long-term memory per agent and tenant (facts, preferences, summaries). Tenant-isolated; no cross-tenant retrieval.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
agent_id	uuid	N		FK → agent.agent_registry(id)
memory_kind	varchar(12)	N		CHECK (memory_kind IN ('working','fact','summary'))
key	varchar(120)	N		
content	jsonb	N		PII-minimised
expires_at	timestamptz	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

UNIQUE (tenant_id, agent_id, memory_kind, key). Optional vector column (pgvector) added only after retrieval design is validated.

agent.agent_logs [P2 · Agent]

Append-only agent action log: initiator, capability, arguments, decision, approval, result, timestamp (Agent Governance).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
agent_id	uuid	N		FK → agent.agent_registry(id)
task_id	uuid	Y		FK → agent.agent_tasks(id)
execution_id	uuid	Y		FK → agent.agent_executions(id)
level	varchar(6)	N	'info'	CHECK (level IN ('debug','info','warn','error'))
tool_name	varchar(80)	Y		
arguments	jsonb	Y		
decision	varchar(20)	Y		
approval_ref	uuid	Y		FK → agent.agent_decisions(id)
result	jsonb	Y		
correlation_id	uuid	N		
created_at	timestamptz	N	now()	append-only

Partition monthly.

5A · Schema audit — audit, security and compliance

audit.audit_logs [P0 · Audit]

Immutable cross-cutting audit (NFR-AUD-01/02/04). Written in the same transaction as the outbox event. Append-only; the app role has INSERT/SELECT only.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK (with occurred_at, partitioning)
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
occurred_at	timestamptz	N	now()	partition key
actor_type	varchar(12)	N		CHECK (actor_type IN ('human','automation','ai','agent','system'))
actor_id	uuid	Y		user or service principal
action	varchar(60)	N		e.g. membership.approve, attendance.correct
resource_type	varchar(40)	N		
resource_ref	varchar(140)	N		ERP docname or uuid
before_data	jsonb	Y		redacted snapshot
after_data	jsonb	Y		
reason	text	Y		
request_context	jsonb	N	'{}'	ip, user agent, route
correlation_id	uuid	N		
prev_hash	char(64)	Y		optional hash chain (tamper evidence)
row_hash	char(64)	Y		

PK (id, occurred_at). PARTITION BY RANGE (occurred_at) monthly. INDEX (tenant_id, occurred_at DESC), (tenant_id, resource_type, resource_ref), (correlation_id).

audit.change_history [P1 · Audit]

Field-level change history for governed configuration/identity entities (plans mirrored config, roles, settings, templates).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
entity_type	varchar(60)	N		
entity_id	varchar(140)	N		
field_name	varchar(80)	N		
old_value	jsonb	Y		
new_value	jsonb	Y		
changed_by	uuid	Y		
audit_log_id	uuid	Y		soft ref to audit_logs.id
changed_at	timestamptz	N	now()	

Append-only. INDEX (tenant_id, entity_type, entity_id, changed_at DESC).

audit.activity_events [P1 · Audit]

Lightweight user-activity tracking (login, view of sensitive record, export) for support and insider-risk review.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
user_id	uuid	Y		
activity	varchar(60)	N		login, view_member, export_report...
resource_ref	varchar(140)	Y		
metadata	jsonb	N	'{}'	
occurred_at	timestamptz	N	now()	
correlation_id	uuid	Y		

Partition monthly; shortest retention.

audit.security_events [P0 · Audit]

Security-relevant events: tenant-boundary denials, failed logins, lockouts, replayed webhooks, signature failures, permission denials (BR-GEN-02). Alert on repetition.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	Y		claimed/resolved tenant (may be NULL for unknown host); no FK to keep record even if tenant removed
event_type	varchar(40)	N		tenant_denied, login_failed, webhook_sig_invalid, rate_limited...
severity	varchar(8)	N	'info'	CHECK (severity IN ('info','warning','critical'))
actor_id	uuid	Y		
ip_address	inet	Y		
route	varchar(200)	Y		
detail	jsonb	N	'{}'	
occurred_at	timestamptz	N	now()	
correlation_id	uuid	Y		

Partition monthly. INDEX (tenant_id, event_type, occurred_at DESC), (ip_address, occurred_at DESC).

audit.data_retention_policies [P1 · Configuration]

Declarative retention per table/class, per tenant override within legal limits; read by cleanup and archival jobs.

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	Y		NULL = platform default
target	varchar(80)	N		schema.table or data class
retention_days	integer	N		
action	varchar(10)	N		CHECK (action IN ('delete','archive','anonymise'))
legal_hold	boolean	N	false	
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

UNIQUE (COALESCE(tenant_id,zero), target).

audit.data_subject_requests [P2 · Audit]

Consent/erasure/access requests handling (Indian data protection alignment; legal review pending).

Column	Data type	Null	Default	Constraint / FK / notes
id	uuid	N	gen_random_uuid()	PK
tenant_id	uuid	N		FK → platform.tenants(id); RLS key
subject_erp_ref	varchar(140)	N		
request_type	varchar(12)	N		CHECK (request_type IN ('access','erasure','correction','consent_withdraw'))
status	varchar(12)	N	'open'	CHECK (status IN ('open','in_progress','done','rejected'))
handled_by	uuid	Y		FK → iam.users(id)
due_at	timestamptz	Y		
outcome	text	Y		
created_at	timestamptz	N	now()	
updated_at	timestamptz	N	now()	set by trigger

Part B — ERP DocTypes (custom app wellness_studio)

DocTypes are stored by Frappe as tables named “tab<DocType>”. Frappe adds name (primary key), owner, creation, modified, modified_by, docstatus and idx automatically. “Req” is shown in the Null column as Y = required (NOT NULL semantics enforced by DocType validation) and N = optional. Tenant, studio and branch are mandatory on every business DocType; a permission-query hook restricts every read/write to the tenant supplied by the integration call. Child tables carry parent, parentfield and parenttype automatically.

ERP.Wellness Tenant Scope [P0 · ERP DocType]

Read-only ERP mirror of tenant identity (code → ERPNext Company) used only by permission hooks and DocType links. App DB platform.tenants is the authority; mirror written by onboarding routine. Naming: name = tenant_code.

Column	Data type	Null	Default	Constraint / FK / notes
tenant_code	Data	N		unique; equals platform.tenants.code
tenant_uuid	Data (36)	N		platform.tenants.id
company	Link (Company)	N		ERPNext Company for this tenant
tier	Select	Y		basic/premium/ultra (informational)
is_active	Check	Y	1	

ERP.Wellness Studio Scope [P0 · ERP DocType]

ERP mirror of platform.studios (id + name) for linking session-level DocTypes.

Column	Data type	Null	Default	Constraint / FK / notes
tenant	Link (Wellness Tenant Scope)	N		
studio_uuid	Data (36)	N		platform.studios.id
studio_name	Data	N		

ERP.Wellness Branch Scope [P0 · ERP DocType]

ERP mirror of platform.branches for branch-scoped records.

Column	Data type	Null	Default	Constraint / FK / notes
tenant	Link (Wellness Tenant Scope)	N		
studio	Link (Wellness Studio Scope)	N		
branch_uuid	Data (36)	N		platform.branches.id
branch_name	Data	N		

ERP.Member [P0 · ERP DocType]

System-of-record member profile; one record per person per tenant (B2B-MEM-01/02). Naming: MEM-NNNNN

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		MEM-NNNNN system generated (BR-GEN-04)
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		
branch	Link (Wellness Branch Scope)	Y		
first_name	Data	N		
last_name	Data	Y		
email	Data (Email)	N		unique per tenant (case-insensitive) — DB unique index (tenant, lower(email))
phone	Data (Phone, E.164)	N		unique per tenant
date_of_birth	Date	Y		must be in past
gender	Select	Y		female/male/other/undisclosed
join_date	Date	N	Today	
status	Select	N	active	active / inactive / frozen / archived (archive instead of delete)
emergency_contact_name	Data	Y		
emergency_contact_phone	Data	Y		
health_notes	Small Text	Y		optional; encrypted field; hidden from Trainer/Front Desk by field permlevel
photo	Attach Image	Y		tenant-prefixed path
source_lead	Link (Lead)	Y		retained on conversion (BR-CRM-02)
app_user_uuid	Data (36)	Y		iam.users.id (identity mapping; reference only)

Naming series MEM-.##### per tenant. Unique: (tenant,phone), (tenant,email). Index: (tenant,status), (tenant,last_name,first_name). Members never deleted; archived (BR-GEN-03).

ERP.Membership Plan [P0 · ERP DocType]

Studio-defined member plan (not a SaaS tier — BR-MSP-01). Edits audited; do not alter existing memberships (BR-MSP-06).

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		PLN-NNNN
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		
branch	Link (Wellness Branch Scope)	Y		
plan_name	Data	N		
description	Small Text	Y		
price	Currency (INR)	N	0	>= 0
validity_days	Int	N		> 0
max_bookings_per_period	Int	Y		NULL = unlimited
booking_limit_period	Select	Y		week / month / validity
requires_approval	Check	Y	0	drives Pending Approval state
is_active	Check	N	1	deactivate, never delete if used
allowed_class_types	Table (Plan Class Entitlement)	Y		plan-to-class entitlement as configuration

Unique (tenant, plan_name).

ERP.Plan Class Entitlement [P0 · Child table]

Child of Membership Plan: which class types a plan entitles (B2B-CLS-02).

Column	Data type	Null	Default	Constraint / FK / notes
class_type	Link (Class Type)	N		
sessions_per_period	Int	Y		optional per-type limit

ERP.Membership [P0 · ERP DocType]

Member-plan assignment with state machine Pending Approval → Active → Expired / Suspended / Cancelled. Payment status and approval status are independent (BR-MSP-02). Naming: MSP-NNNNN

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		MSP-NNNNN
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		
branch	Link (Wellness Branch Scope)	Y		
member	Link (Member)	N		must exist (BR-MSP-03)
plan	Link (Membership Plan)	N		
plan_snapshot	JSON	N		price/validity/entitlements at assignment time (protects BR-MSP-06)
start_date	Date	N		
end_date	Date	N		> start_date
status	Select	N	Pending Approval	Pending Approval / Active / Expired / Suspended / Cancelled
approval_status	Select	N	pending	pending / approved / rejected / not_required
approved_by	Link (User)	Y		
approved_on	Datetime	Y		
payment_status	Select	N	unpaid	unpaid / pending / paid / refunded — independent of approval
renewal_of	Link (Membership)	Y		manual renewal chain
bookings_used	Int	N	0	counter, updated atomically with bookings

Index: (tenant,member,status), (tenant,status,end_date) for expiry scheduler and expiring-soon list. Expired set by ERP scheduler; emits membership.expired callback.

ERP.Class Type [P0 · ERP DocType]

Configurable class/service type (Dance, Yoga, Zumba seed; Pilates etc. added without code).

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		CT-NNNN
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		

Column	Data type	Null	Default	Constraint / FK / notes
branch	Link (Wellness Branch Scope)	Y		
type_name	Data	N		
description	Small Text	Y		
color	Color	Y		
default_capacity	Int	Y		overrides tenant default 30
is_active	Check	N	1	archived not deleted when used

Unique (tenant,type_name).

ERP.Class Schedule [P0 · ERP DocType]

Recurring or one-off schedule template that expands into Class Sessions (recurrence expander).

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		SCH-NNNNN
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		
branch	Link (Wellness Branch Scope)	Y		
class_type	Link (Class Type)	N		
trainer	Link (Wellness Staff)	Y		
start_time	Time	N		
duration_minutes	Int	N		> 0
recurrence_rule	Data	Y		RFC-5545 RRULE subset; NULL = one-off
valid_from	Date	N		
valid_to	Date	Y		
capacity	Int	Y		NULL = class type/tenant default
is_active	Check	N	1	

ERP.Class Session [P0 · ERP DocType]

Concrete scheduled session; capacity and booking counters are the locking target for atomic booking (NFR-PRF-03). Naming: SES-NNNNNN

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		SES-NNNNNN
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		
branch	Link (Wellness Branch Scope)	Y		
class_type	Link (Class Type)	N		
schedule	Link (Class Schedule)	Y		
session_date	Date	N		
start_datetime	Datetime	N		
end_datetime	Datetime	N		> start
trainer	Link (Wellness Staff)	Y		
capacity	Int	N		resolved at creation from config (session > schedule > class type > tenant default)
confirmed_count	Int	N	0	updated only inside atomic booking method (row lock)
waitlist_count	Int	N	0	
status	Select	N	scheduled	scheduled / cancelled / completed

CHECK confirmed_count <= capacity. Reducing capacity below confirmed_count blocked. Index: (tenant,session_date,status), (tenant,trainer,start_datetime).

ERP.Booking [P0 · ERP DocType]

Atomic booking record; status Booked / Waitlisted / Cancelled with queue position (B2B-BKG-01/02). Naming: BKG-NNNNNN

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		BKG-NNNNNN
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		
branch	Link (Wellness Branch Scope)	Y		

Column	Data type	Null	Default	Constraint / FK / notes
member	Link (Member)	N		
class_session	Link (Class Session)	N		
membership	Link (Membership)	N		membership used for eligibility
status	Select	N		Booked / Waitlisted / Cancelled / Promoted (transitional)
queue_position	Int	Y		waitlist order
booked_on_behalf_by	Link (User)	Y		Admin/Front Desk/Trainer with grant
booking_source	Select	N	self	self / on_behalf / trainer_assisted / ai_flow (P2)
cancelled_on	Datetime	Y		
cancel_reason	Small Text	Y		
idempotency_key	Data	Y		write idempotency

Unique (tenant, member, class_session) WHERE status IN (Booked, Waitlisted) — duplicate protection. Cancellation cutoff (default 4 h) enforced in ERP method.

ERP.Attendance Record [P0 · ERP DocType]

One attendance record per member per session/event session (BR-ATT-01/04). Naming: ATT-NNNNNN

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		ATT-NNNNNN
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		
branch	Link (Wellness Branch Scope)	Y		
member	Link (Member)	N		
session_kind	Select	N		class_session / event
class_session	Link (Class Session)	Y		exactly one of class_session / event
event	Link (Event)	Y		
booking	Link (Booking)	Y		
method	Select	N		qr / trainer / front_desk / admin_correction
status	Select	N	present	present / absent / late
marked_by	Link (User)	Y		
marked_on	Datetime	N	Now	
corrections	Table (Attendance Correction)	Y		immutable history

Unique (tenant, member, class_session) and (tenant, member, event). CHECK exactly one of class_session/event set. Trainer permitted only for assigned sessions.

ERP.Attendance Correction [P0 · Child table]

Child table: immutable correction history — actor, timestamp, previous value, new value, reason (BR-ATT-03). No edit/delete permission at any role.

Column	Data type	Null	Default	Constraint / FK / notes
corrected_by	Link (User)	N		actor
corrected_on	Datetime	N	Now	
previous_value	Data	N		
new_value	Data	N		
reason	Small Text	N		

ERP.Event [P0 · ERP DocType]

Workshop/studio/public event with capacity, window, audience, price (BR-EVT-01..04).

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		EVT-NNNNN
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		
branch	Link (Wellness Branch Scope)	Y		
title	Data	N		
category_code	Data	N		validated against config.event_categories
description	Text Editor	Y		
start_datetime	Datetime	N		
end_datetime	Datetime	N		
capacity	Int	N		

Column	Data type	Null	Default	Constraint / FK / notes
registration_opens	Datetime	N		
registration_closes	Datetime	N		<= start
audience	Select	N	public	public / members / prospects
is_paid	Check	N	0	
price	Currency (INR)	Y		required if is_paid
trainer	Link (Wellness Staff)	Y		
status	Select	N	draft	draft / published / cancelled / completed

ERP.Event Registration [P0 · ERP DocType]

Registration by member or external registrant; paid registration confirmed only after verified payment (B2B-EVT-02).

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		REG-NNNNNN
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		
branch	Link (Wellness Branch Scope)	Y		
event	Link (Event)	N		
member	Link (Member)	Y		exactly one of member / lead
lead	Link (Lead)	Y		external registrant (Workshop Registrant)
status	Select	N		Hold / Confirmed / Cancelled / Expired
payment	Link (Payment Record)	Y		
hold_expires_at	Datetime	Y		seat released after expiry (default 15 min)
registered_on	Datetime	N	Now	

Unique (tenant,event,member) and (tenant,event,lead). Capacity check inside atomic registration method.

ERP.Lead [P0 · ERP DocType]

Prospect/registrant record; never a Member before conversion (BR-CRM-01/02).

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		LEAD-NNNNN
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		
branch	Link (Wellness Branch Scope)	Y		
full_name	Data	N		
phone	Data (Phone)	N		duplicate flagged/merged (BR-CRM-03)
email	Data (Email)	Y		
source	Select	N		immutable after creation: landing_page, ad, walk_in, referral, enquiry_ai, event
campaign	Data	Y		editable by Admin only
status	Select	N	new	new / contacted / qualified / converted / lost
consent_given	Check	N	0	
consent_recorded_on	Datetime	Y		
converted_member	Link (Member)	Y		set on conversion
converted_on	Datetime	Y		
merged_into	Link (Lead)	Y		duplicate merge target
activities	Table (Lead Activity)	Y		

Index: (tenant,phone), (tenant,email), (tenant,status,creation), (tenant,source).

ERP.Lead Activity [P0 · Child table]

Child table: follow-up notes and status changes retained after conversion.

Column	Data type	Null	Default	Constraint / FK / notes
activity_type	Select	N		note / call / message / status_change
note	Small Text	Y		
activity_on	Datetime	N	Now	
by_user	Link (User)	Y		

ERP.Payment Record [P0 · ERP DocType]

Internal payment record (BR-PAY-04). Only verified server-side confirmation sets status Paid; posted after Razorpay webhook verification.

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		PAY-NNNNNN (internal id)
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		
branch	Link (Wellness Branch Scope)	Y		
member	Link (Member)	Y		
lead	Link (Lead)	Y		external event registrant
purpose	Select	N		membership / event / other
membership	Link (Membership)	Y		
event_registration	Link (Event Registration)	Y		
provider	Select	N		razorpay / manual_cash / manual_upi / manual_other
provider_reference	Data	Y		gateway payment_id/order_id; unique when set
amount	Currency	N		>= 0
currency	Link (Currency)	N	INR	
status	Select	N	Created	Created / Pending / Paid / Failed / Refunded
paid_on	Datetime	Y		
recorded_by	Link (User)	Y		manual entries require actor + reference (B2B-PAY-02)
manual_reference	Data	Y		
erp_invoice	Link (Sales Invoice)	Y		Phase 2 (ERPNext accounting/GST)

Unique (tenant, provider, provider_reference) WHERE provider_reference IS NOT NULL → idempotent confirmations (NFR-REL-01). Status transitions validated (no Failed → Paid without new verified event).

ERP.Wellness Staff [P0 · ERP DocType]

Trainer/staff ERP record: studio scope, class assignments, active flag. Credentials/roles live in App DB (iam).

Column	Data type	Null	Default	Constraint / FK / notes
name (ID)	Data	N		STF-NNNN
tenant	Link (Wellness Tenant Scope)	N		mandatory; injected by adapter; permission-query hook filters on it
studio	Link (Wellness Studio Scope)	Y		
branch	Link (Wellness Branch Scope)	Y		
full_name	Data	N		
email	Data (Email)	N		
phone	Data (Phone)	Y		
staff_role	Select	N		owner/admin/front_desk/trainer (informational; RBAC lives in iam)
is_trainer	Check	N	0	
assigned_class_types	Table (Staff Class Assignment)	Y		trainer class-type assignments
is_active	Check	N	1	history retained on deactivation
app_user_uuid	Data (36)	Y		iam.users.id

ERP.Staff Class Assignment [P0 · Child table]

Child of Wellness Staff: class types a trainer is assigned to.

Column	Data type	Null	Default	Constraint / FK / notes
class_type	Link (Class Type)	N		

6. Multi-Tenant Database Strategy

6.1 Tenant Isolation Model

Layer	Mechanism
PostgreSQL (chosen)	Shared database, shared schemas, tenant_id on every row. Row-level security (RLS) policies keyed to current_setting('app.tenant_id'). FORCE ROW LEVEL SECURITY so even the table owner is subject to policy. The application role is NOT a superuser and lacks BYPASSRLS.
ERP	Single Frappe site for beta; each tenant is an ERPNext Company; every custom DocType has a tenant link; permission-query hooks + write validation in the custom app restrict to the tenant passed by the adapter; adapter always injects tenant (Architecture §6.1). Alternative site-per-tenant is supported by routing.
Cross-store	The same tenant UUID/code is carried in both stores (Wellness Tenant Scope mirror), in every outbox event and in every audit row.
Files / logs	Tenant-prefixed object-storage paths; logs tagged with tenant_id and correlation_id.

6.2 Tenant Identification Strategy

- Browser requests: host → platform.tenant_domains(host) → tenant_id (AD11). Resolved in middleware; result bound to the session/JWT tenant claim; mismatch → 404-style denial + security_events row.
- Service calls (n8n, ERP callbacks): tenant_id in the signed event envelope; verified against the service principal's allowed tenant.
- AI and agents: run under the invoking user's or the agent principal's tenant; retrieval sources filtered by tenant; no cross-tenant index.
- Per-request database context: the API opens a transaction and executes SELECT set_config('app.tenant_id', $1, true) (equivalent to SET LOCAL) before any query. A missing setting makes policies evaluate to false — fail closed.
- The tenant resolver function platform.resolve_tenant(host) is SECURITY DEFINER with a narrow grant so it can run before a tenant context exists.

6.3 Shared Data Model

Data	Sharing model
Platform catalogues: subscription_tiers, tier_features, permissions, system roles (tenant_id NULL), report_definitions (standard), agent_registry, ai_prompt_versions, ai_eval_runs	Global rows readable by all tenants (read-only for tenants), writable only by platform administrators/migrations.
Tenant data: everything else	Strictly tenant-private.
Security/audit rows with nullable tenant_id (unknown host, platform events)	Visible to platform-admin role only.
Cross-tenant analytics	None in Phase 1. If introduced, only via anonymised platform-level aggregates computed by a privileged job, never by tenant-scoped role.

6.4 Tenant Configuration Storage

Resolution order: platform defaults → tier entitlements (platform.tier_features) → tenant overrides (platform.tenant_feature_flags, config.tenant_settings). Presentation and operational configuration (branding, templates, timings, flags, categories) live in the App DB; business-record configuration (plans, class types, capacities on types/sessions) lives in the ERP. The Config service caches reads per tenant and invalidates on write; every change writes audit.audit_logs and audit.change_history.

Config item	Store	Table
Branding	App DB	config.tenant_branding
Default capacity 30, cancellation cutoff 4 h, hold minutes, drop-in	App DB	config.tenant_settings
Reminder timing and template mapping	App DB	config.notification_rules / notification_templates
Feature flags / AI enablement by tier	App DB	platform.tier_features + tenant_feature_flags
Event categories	App DB	config.event_categories
Membership plans, class types, per-session capacity	ERP	Membership Plan, Class Type, Class Schedule/Session

6.5 Future Scaling Strategy

Trigger	Step (no caller change)
Table growth on shared schema	Hash-sub-partition largest partitioned tables by tenant_id; add BRIN on time columns.
Noisy or enterprise tenant	ERP: dedicated Frappe site via tenant_erp_routes. PG: move tenant to dedicated database/cluster; connection routed by tenant→DSN map (same schema, same migrations).
Data residency / white-label	tenants.data_region + routing; custom domains via tenant_domains.kind = custom.
Read load	Read replica for reporting, audit views, kpi_snapshots.

Trigger	Step (no caller change)
Multi-branch / franchise	Branch scope already in schema; add location-aware roles and reporting.
AI/notification load	Move outbox dispatcher and AI jobs to workers; queue-based ingestion; partition pruning keeps hot partitions small.

6.6 Why this approach is suitable

- Fits a 5-day delivery with a six-person team: one schema set, one migration pipeline, one connection pool.
- RLS gives database-enforced defence in depth for the highest-rated risk (R-09 cross-tenant exposure) on top of the tenant guard and repository scoping.
- Isolation cost is low for small studios (hundreds of members each); database-per-tenant would multiply operations for no benefit at this stage.
- The escape hatches (routing tables, partitioning, dedicated site/database) are already designed, so growth is a configuration change rather than a redesign (NFR-SCL-03).
- The same tenant model is used by the ERP, App DB, AI and automation, so isolation tests (S2) can be written once and run against every layer.

7. ERP Data Model

7.1 ERP-related entities

Group	Entities
ERP master (system of record)	Member, Membership Plan (+Plan Class Entitlement), Membership, Class Type, Class Schedule, Class Session, Booking, Attendance Record (+Attendance Correction), Event, Event Registration, Lead (+Lead Activity), Payment Record, Wellness Staff (+Staff Class Assignment); ERPNext Company, Customer/Sales Invoice/GL (Phase 2)
ERP scope mirrors (read-only)	Wellness Tenant Scope, Wellness Studio Scope, Wellness Branch Scope
Application master (App DB)	Tenants, studios, branches, tiers, flags; users, credentials, roles, permissions, grants, sessions; branding, settings, templates, rules, categories; audit; outbox; notification records; AI/agent data
Shared / linked records	Identity link (iam.user_erp_links ↔ Member/Staff), event subject references (ops.outbox_events.subject_ref), notification recipient refs, feedback refs to Class Session, audit resource refs, AI recommendation refs

7.2 Data Ownership

Record	Owner	Notes
Member, Membership, Plan, Class*, Booking, Attendance, Event*, Lead, Payment, Invoice	ERP	Authoritative; App DB may hold ID and a labelled snapshot only
Tenant/Studio/Branch	App DB	ERP mirrors written by onboarding routine
Users, roles, permissions, grants, sessions, MFA	App DB	Members and staff logins are not ERP users; only the mapping is shared
Notification templates, timings, delivery records	App DB	ERP stores none
Audit (incl. AI)	App DB	ERP additionally keeps Attendance Correction child history (business fact) and standard Frappe version log
Payment posting to accounting	ERP	After verified Razorpay confirmation

7.3 Master Data Strategy

- One master per fact. There is no bidirectional entity sync: all ERP-owned reads/writes are synchronous call-throughs at request time via the ERP adapter; no shadow copy of business truth (Architecture §7.3).
- IDs are generated by the ERP for ERP records and by PostgreSQL for App DB records; clients never supply identifiers (BR-GEN-04).
- Reference by ID: App DB rows store ERP docnames (member_erp_ref, subject_ref, resource_ref). Display/audit snapshots are labelled and time-stamped, never used for decisions.
- Identity mapping: iam.user_erp_links resolves user ↔ Member/Staff; member account linking matches on verified phone within the tenant, preventing duplicates.
- Configuration master split: business-record configuration in ERP; presentation/operational configuration in App DB.
- Optional short-TTL read cache (dashboard aggregates and configuration only) — never for booking availability, eligibility or payment status.

7.4 Synchronisation Rules

Flow	Direction	Rule
Application intent → ERP write	API → ERP	Adapter calls whitelisted atomic methods with idempotency key, tenant, correlation ID; audit + outbox written only after ERP success (same PG transaction).
ERP scheduler/hook events	ERP → API	Signed callbacks (expiry, hold timeout) validated by API, recorded in ops.webhook_inbox, audited, then written to ops.outbox_events — one event path.
Payment confirmation	Razorpay → API → ERP	Signature-verified webhook → payment Paid in ERP → outbox payment.confirmed. Delayed webhook leaves Pending; status-fetch reconciliation for stale Pending.
Tenant onboarding	App DB → ERP	Onboarding routine creates tenant/studio/branch in PG, then mirror scope DocTypes and ERPNext Company; failures roll back the tenant to status = onboarding.
KPI aggregates	ERP → App DB	Scheduled pull into reporting.kpi_snapshots (advisory).
Nightly reconciliation	Both	Detect orphaned refs, outbox events without outcome, stale Pending payments; write ops.reconciliation_findings and admin alerts; no automatic mutation of business data.
ERP unavailable	—	Adapter returns explicit “temporarily unavailable”; no cached data is presented as confirmed (NFR-AVL-03).

7.5 Conflict Resolution Strategy

Situation	Resolution
Same business fact differs between ERP and App DB snapshot	ERP wins; snapshot refreshed or flagged by reconciliation.
Concurrent writes to a session/seat	Row lock in the ERP atomic method; exactly one confirmation; others get structured reason (full / duplicate / ineligible).
Concurrent updates to an App DB configuration row	Optimistic concurrency (version column); loser receives 409 and re-reads.

Situation	Resolution
Duplicate webhook or event delivery	Idempotent by provider_event_id / event id / dedupe_key; second application is a no-op.
Orphaned reference (ERP doc missing)	Reported in reconciliation findings; never auto-deleted; Owner/Admin resolves.
Duplicate person (member/lead)	Rejected on unique email/phone; leads flagged or merged with merged_into pointer; history retained.
Tenant mismatch across stores	Hard failure: request denied, security_events row, alert.
Adapter/custom-app version mismatch	Startup compatibility check; deploy custom app before adapter; write ops.erp_contract_versions.

7.6 Record classification

Class	Records
ERP Master Records	Member, Lead, Membership Plan, Membership, Class Type, Class Schedule, Class Session, Booking, Attendance Record + Correction, Event, Event Registration, Payment Record, Wellness Staff, (Phase 2) Sales Invoice/GL entries
Application Master Records	Tenant, Studio, Branch, Subscription Tier, Feature Flags, Domain, User, Role, Permission, Grant, Session, Branding, Settings, Event Category, Notification Template/Rule, FAQ, Report Definition, Agent Registry
Shared Records (reference by ID across stores)	user_erp_links; outbox subject refs; notification recipient refs; consent subject refs; AI recommendation and feedback member/session refs; audit resource refs; ERP scope mirrors of tenant/studio/branch

8. AI Data Model

Schema ai (Section 5A). Design rules: AI never touches business tables directly (BR-AI-01); conversation context is temporary (BR-AI-07); one immutable audit record per tool call (NFR-AUD-03); PII minimised before storage and prompts (NFR-SEC-05); AI features gated by tier flags (BR-AI-11).

Table	Purpose	Relationships	Storage strategy
ai_sessions	Session envelope: principal, assistant type, permissions snapshot, expiry	tenant, user; 1:N conversations, pending actions, tool-call audit, usage, recommendations	Hot OLTP table; small rows; status/expiry indexed; sessions auto-expire.
ai_conversations	Per-turn messages for context	session_id FK; optional tool_call_id	Redacted text only; purge_after TTL; not partitioned in P0 (short-lived), monthly partition if volume grows.
ai_pending_actions	Propose → confirm store with fixed arguments and hash	session, user; 0..N tool-call audit	Small OLTP; unique active proposal per session+tool recommended; expires quickly.
ai_tool_call_audit	Immutable per-call audit	session, pending action, actor	Append-only, monthly RANGE partitions, revoke UPDATE/DELETE; mirrored summary in audit.audit_logs.
ai_recommendations	Candidate set and ranking generated for a member; acceptance tracking	session; member_erp_ref; accepted_session_ref → ERP Class Session	OLTP with jsonb candidates; expires after use window; supports acceptance-rate KPI.
ai_feedback_submissions	Raw member comment (record of truth)	member_erp_ref, class_session_ref (ERP)	OLTP; restricted column-level access (Owner/Admin only).
ai_feedback_analysis	Sentiment, topics, theme, confidence, model/prompt version	submission_id FK	Separate table so classification can be re-run per prompt version; human_label supports ≥80% agreement sampling.
ai_feedback_theme_summaries	Weekly/per-session aggregate for admins	scope_ref (session/class type/week)	Derived, regenerable; aggregate-only visibility for trainers.
ai_usage_logs	Token and cost metering per call	session_id; tenant	Append-only, monthly partitions; rolled up to a daily materialised view for quota checks.
ai_prompt_versions	Traceability of prompt template versions	referenced by audit/analysis by version string	Tiny reference table; prompts themselves are versioned in Git.
ai_eval_runs	Evaluation harness results (85%/95%/zero-false-success)	none (platform level)	Small; keeps history per git_sha and model.

8.1 Retention strategy

Table	Retention	Action at end
ai_sessions	30 days after expiry (configurable)	Delete (metadata only remains in audit)
ai_conversations	7–30 days (tenant policy; default 14 days); enquiry (visitor) 7 days	Delete; content never becomes business data
ai_pending_actions	90 days after decision	Archive with audit reference then delete
ai_tool_call_audit	Follows audit policy: 24 months online, then archive (legal review pending)	Detach partition → archive to cold storage
ai_recommendations	90 days	Delete (acceptance counts kept in kpi_snapshots)
ai_feedback_submissions / analysis	24 months or until member erasure request	Anonymise member_erp_ref; keep aggregates
ai_usage_logs	13 months detail; daily rollup kept 36 months	Drop partition
ai_eval_runs / prompt_versions	Indefinite	—

8.2 Storage notes

- No embeddings in Phase 1: Enquiry answers use full-text retrieval over config.faq_entries (GIN tsvector) plus public class/plan/event reads through services. A pgvector column may be added later behind the same tenant-filtered retrieval interface.
- Model-agnostic: model_id, provider and prompt_version are recorded on audit, analysis and usage rows so results are reproducible across provider changes (R-10).
- Arguments and results in audit are bounded (size limit) and redacted (phone, email, health notes masked) before insert.

9. Agent Data Model

Agents (CRM, Operations P2; Marketing, Advertising, Growth, Studio Insights P3; Session Summary Future) run as background workers using the same tool registry, LLM gateway, audit and approval pattern as the Assistant. The schema is created empty in the first migration set so agents are added without redesign; tables are P2 build-time.

Table	Purpose	Relationships	Future scalability notes
agent_registry	Catalogue of agents with domain, phase, default approval mode	1:N tenant configs, tool grants, tasks, memory	Global reference; adding an agent = insert row + tool grants; no DDL.
agent_tenant_configs	Per-tenant enablement, schedule, service principal	agent, service principal (iam)	Gated by tier flag; supports per-tenant schedules and quotas.
agent_tool_grants	Domain-scoped tool allow-list with permission ceiling	agent; permission code	Enforces “each agent gets only its domain tools”; re-checked at execution.
agent_tasks	Unit of work from event, schedule or user	agent, on_behalf_of user	Queue semantics: index (status, priority, created_at) + SKIP LOCKED workers; partition when large.
agent_executions	Execution attempts per task	task 1:N	Retry history; token and latency stats feed cost control.
agent_decisions	Proposal + human approval decision (approval queue)	task, execution, decider	Same pattern as ai_pending_actions (frozen args + hash); UI added in Phase 2.
agent_memory	Scoped working/fact/summary memory	agent; tenant	Tenant-isolated; TTL for working memory; optional vector index later.
agent_logs	Append-only agent action log	agent, task, execution, decision	Monthly partitions; shipped to log store; feeds Studio Insights and audit views.

9.1 Governance constraints encoded in schema

- Each agent has a service principal scoped to one tenant (agent_tenant_configs.service_principal_id) and one domain toolset (agent_tool_grants).
- Sensitive actions must create an agent_decisions row with requires_approval = true; execution is permitted only when status = approved and proposal_hash matches.
- Agents propose; they never write business data directly — writes go through governed services and produce ordinary audit + outbox rows with actor_type = agent.
- Cross-agent collaboration uses ops.outbox_events, not agent-to-agent calls.
- Advertising/Marketing: CHECK-level default_approval = required in registry; no credentials with publish rights are stored (secret_ref only, and none granted to agents).

10. Automation Data Model

Schema notify plus ops.outbox_events. n8n orchestrates and is never a source of truth (AD07): all durable state — dedupe, consent, status, failure — is stored here through the API, not in n8n variables.

Table	Purpose	Relationships	Retention rules
workflow_executions	Trace of each n8n run reported to API	outbox event; audit trail; correlation id	Partitioned monthly; 90 days online for succeeded, 12 months for failed/dead; then archive.
notification_messages	Notification Log per event+recipient+template with dedupe key	outbox event, template, ERP recipient ref	24 months (business record of communication); recipient_address_enc nulled after 90 days.
whatsapp_messages	Provider-level send/mock record	notification 1:N; delivery events	12 months; payload_redacted only.
notification_delivery_events	Status callbacks appended per message	whatsapp message; webhook inbox	12 months with parent.
email_logs (P2)	Transactional email records	notification	12 months.
automation_audit_trails	Decision steps (consent, dedupe, retry, alert)	workflow execution; notification	24 months (audit class).
communication_consents / suppression_list	Consent evidence and suppression	ERP subject ref	Life of relationship + 3 years, legal hold aware; never deleted while messages are sendable.
admin_alerts	Failure visibility tasks	notification/subject	12 months after resolution.
ops.outbox_events	Reliable event hand-off	notifications, executions	Delivered rows 30 days then archive/drop; dead rows kept until resolved.
ops.webhook_inbox	Raw inbound webhook receipts	delivery events	Payment webhooks 24 months; others 90 days.

10.1 Idempotency and consent flow in data terms

- Idempotency key: dedupe_key = event_id + recipient + template, UNIQUE (tenant_id, dedupe_key). The API returns the existing row for a repeat request; n8n cannot create a second send (BR-NTF-03).
- Consent: before releasing a send, the API reads the latest communication_consents row for (subject, channel, category); marketing without opt_in → status skipped_consent + automation_audit_trails decision.
- Mock/live: notification_messages.mode and whatsapp_messages.mock record which mode was used; a would-send record has recipient hash, template, time (BR-NTF-04). Live credentials swap by configuration only.
- Failure: bounded retries increment attempts; final failure sets status = failed and inserts admin_alerts; the originating business transaction is untouched (BR-NTF-02).
- Time-based reminders: n8n schedules call the API “what is due” query, which reads ERP data and config.notification_rules, returns deduplicated recipients, and inserts queued notification_messages rows in one step.

11. Audit & Compliance Tables

Table	Purpose	Structure (key columns)	Retention policy
audit.audit_logs	Immutable record of approvals, payment changes, attendance corrections, role/permission/config changes, deletions, boundary denials, every AI tool call summary, every automated send	tenant_id, occurred_at, actor_type (human/automation/ai/agent/system), actor_id, action, resource_type/ref, before_data, after_data, reason, request_context, correlation_id, optional hash chain	Monthly partitions. 24 months online; critical history (payments, approvals, attendance corrections, permission changes) archived, never deleted (BR-GEN-03); final periods subject to legal review.
audit.activity_events	Lightweight activity tracking for support/insider-risk (login, sensitive views, exports)	tenant_id, user_id, activity, resource_ref, metadata, occurred_at	90 days online; aggregated counts kept 13 months.
audit.security_events	Tenant-boundary denials, failed logins, lockouts, invalid webhook signatures, rate-limit hits	tenant_id (nullable), event_type, severity, actor_id, ip_address, route, detail, correlation_id	24 months; critical severity archived.
audit.change_history	Field-level history for configuration and identity entities	entity_type, entity_id, field_name, old_value, new_value, changed_by, audit_log_id	Life of entity + 24 months; archived with entity.
ai.ai_tool_call_audit	AI-specific audit (request, tool, arguments, decision, approval, result)	see Section 5A ai	Same as audit_logs.
ERP: Attendance Correction	Immutable attendance correction history (5 fields)	corrected_by, corrected_on, previous_value, new_value, reason	Life of attendance record; archived not deleted.

11.1 Write path and immutability

- The interceptor/service writes audit_logs in the same transaction as ops.outbox_events, after the ERP call succeeds; correlation_id is propagated UI → API → ERP → n8n → WhatsApp callbacks (NFR-AUD-04).
- Append-only enforced by privileges (REVOKE UPDATE, DELETE, TRUNCATE) plus a BEFORE UPDATE OR DELETE trigger raising an exception (Appendix A). Partition detach is performed only by the retention role.
- Optional tamper evidence: prev_hash/row_hash chain per tenant per day, verified by a scheduled job (P1/P2).
- Access: Owner full, Admin limited through read-only views (audit.v_audit_owner, audit.v_audit_admin); export Phase 2.
- PII in before/after snapshots is redacted according to a field allow-list; health notes are never written to audit in clear text.

12. Indexing Strategy

Principles: every composite index used by tenant traffic leads with tenant_id; partial indexes cover hot status subsets; partitioned tables index locally per partition; every FK column used for joins or cascades is indexed; indexes are added from measured plans (pg_stat_statements), not speculatively, beyond those below.

12.1 Primary indexes

Every table has a PRIMARY KEY (uuid) automatically indexed. Composite PKs: role_permissions (role_id, permission_id); audit_logs (id, occurred_at) and other partitioned tables (id, time column); user_credentials (user_id).

12.2 Secondary and composite indexes (App DB)

Table	Index	Purpose
platform.tenant_domains	UNIQUE (host)	Host → tenant resolution on every request
iam.users	UNIQUE (tenant_id, email) WHERE email IS NOT NULL; UNIQUE (tenant_id, phone_e164) WHERE phone_e164 IS NOT NULL; (tenant_id, status)	Login lookup, duplicate prevention, staff lists
iam.auth_identities	UNIQUE (tenant_id, provider, provider_subject)	Google/phone identity lookup and linking
iam.auth_sessions	UNIQUE (refresh_token_hash); (user_id, revoked_at, expires_at)	Refresh rotation and revocation
iam.user_role_assignments	(tenant_id, user_id) ; (tenant_id, role_id)	Permission evaluation per request
iam.permission_grants	partial (tenant_id, user_id) WHERE revoked_at IS NULL	Active grants only
iam.otp_challenges	(tenant_id, phone_e164, expires_at DESC)	Rate limiting and verification
config.tenant_settings	UNIQUE (tenant_id, COALESCE(studio_id,zero), setting_key)	Cached config reads
config.notification_rules	(tenant_id, event_type, is_enabled)	Automation lookup by event type
config.faq_entries	GIN (search_tsv) WHERE status = 'approved'	Enquiry Assistant retrieval
ops.outbox_events	partial (next_attempt_at) WHERE status IN ('pending','failed'); (tenant_id, occurred_at DESC)	Dispatcher polling with SKIP LOCKED; support view
ops.idempotency_keys	UNIQUE (tenant_id, scope, key); (expires_at)	Dedup and cleanup
ops.webhook_inbox	UNIQUE (provider, provider_event_id); (status, received_at)	Replay protection; retry of failed processing
notify.notification_messages	UNIQUE (tenant_id, dedupe_key); (tenant_id, status, created_at DESC); (tenant_id, recipient_erp_ref, created_at DESC)	Idempotency, admin log filter, member communication history
notify.whatsapp_messages	UNIQUE (provider_message_id) WHERE NOT NULL; (notification_id)	Status callback matching
notify.communication_consents	(tenant_id, subject_erp_ref, channel, category, recorded_at DESC)	Latest-consent lookup
notify.workflow_executions	(tenant_id, status, started_at DESC); (outbox_event_id)	Failure visibility
ai.ai_sessions	(tenant_id, user_id, started_at DESC); (expires_at) WHERE status = 'active'	History and expiry sweeps
ai.ai_conversations	UNIQUE (session_id, turn_no); (purge_after)	Context load and TTL purge
ai.ai_pending_actions	(tenant_id, session_id, status); (expires_at) WHERE status = 'proposed'	Confirm/expire flow
ai.ai_tool_call_audit	(tenant_id, occurred/created_at DESC); (session_id); (correlation_id)	Audit review
ai.ai_feedback_*	(tenant_id, class_session_ref); (tenant_id, sentiment, analyzed_at DESC)	Session summaries and trends
agent.agent_tasks	(status, priority, created_at) WHERE status = 'queued'	Worker queue
audit.audit_logs	(tenant_id, occurred_at DESC); (tenant_id, resource_type, resource_ref); (correlation_id)	Audit views and trace lookup
audit.security_events	(tenant_id, event_type, occurred_at DESC); (ip_address, occurred_at DESC)	Alerting on repeated denials
reporting.kpi_snapshots	UNIQUE (tenant_id, studio, kpi_code, period_kind, period_start, dimensions)	Idempotent aggregate upserts; dashboard reads

12.3 ERP DocType indexes (Frappe database indexes / unique constraints)

DocType	Index / constraint	Purpose
Member	UNIQUE (tenant, lower(email)); UNIQUE (tenant, phone); (tenant, status); (tenant, last_name, first_name)	Duplicate control; search and filter
Membership	(tenant, member, status); (tenant, status, end_date)	Eligibility check; expiry scheduler; expiring-soon list
Class Session	(tenant, session_date, status); (tenant, trainer, start_datetime); (tenant, class_type, session_date)	Schedule views; trainer roster; low-attendance reports
Booking	UNIQUE partial (tenant, member, class_session) for Booked/Waitlisted; (tenant, class_session, status, queue_position); (tenant, member, creation)	Duplicate protection; waitlist promotion; member history
Attendance Record	UNIQUE (tenant, member, class_session); UNIQUE (tenant, member, event); (tenant, class_session)	Duplicate check-in; rosters

DocType	Index / constraint	Purpose
Event / Event Registration	(tenant, status, start_datetime); UNIQUE (tenant, event, member); (tenant, event, status)	Event listing; capacity count
Lead	(tenant, phone); (tenant, email); (tenant, status, creation); (tenant, source)	Duplicate detection; pipeline; attribution
Payment Record	UNIQUE (tenant, provider, provider_reference) WHERE NOT NULL; (tenant, member, creation); (tenant, status, creation)	Idempotent confirmation; history; reconciliation

12.4 Search optimisation

- Member/lead search: ERP indexed columns plus prefix search on name/phone/email; if ERP search latency exceeds budget, App DB pg_trgm index on a search projection is added (not a business copy) — decision deferred to Day-1 spike S4.
- FAQ retrieval: generated tsvector with GIN; ranking by ts_rank_cd, tenant filter first.
- Audit and notification search: filter by tenant + time window (partition pruning) + resource/recipient ref; free-text search intentionally not offered.

12.5 Reporting optimisation

- Dashboards and Assistant read the same reporting service; KPIs computed in ERP (aggregate queries) and cached in reporting.kpi_snapshots with computed_at.
- Scheduled hourly/daily aggregation jobs upsert snapshots (unique key gives idempotence); heavy reports use replica or aggregate tables.
- Materialised views: ai.ai_usage_daily, notify.v_delivery_stats (daily counts by status), refreshed concurrently.
- BRIN indexes on time columns of large append-only partitions for range scans of recent history.

13. Data Retention & Archival

13.1 Retention rules

Retention is declarative in audit.data_retention_policies (platform defaults, per-tenant override within legal minimums). Values below are proposed defaults subject to the pending legal review (R-15, NFR-CMP-01).

Data class	Default online retention	After retention
Business records in ERP (members, memberships, bookings, attendance, payments)	Life of tenant relationship; accounting records per statutory period	Archive (status archived); anonymise personal fields on verified erasure while keeping financial facts
Identity (users, roles, grants)	Life of account; deactivated users kept	Soft-deactivate; anonymise PII after erasure request
Auth sessions / OTP challenges	Sessions: expiry + 30 days; OTP: 24 h	Delete
Audit logs / AI audit / security events	24 months online	Archive partitions to cold storage; critical history retained, not deleted
Activity events	90 days	Delete/aggregate
Notification records	24 months	Anonymise recipient hash/ref after 24 months; archive
WhatsApp/email wire records, delivery events	12 months	Drop partition
Outbox events	Delivered: 30 days; dead: until resolved	Drop partition after archive
Idempotency keys	48 hours	Delete
Webhook inbox	Payment 24 months, others 90 days	Archive/Delete
AI conversations	14 days (7 for visitors)	Delete
AI usage logs	13 months	Drop partition (rollup kept)
KPI snapshots	36 months (day), indefinite (month)	Roll up then delete detail
Consent history	Relationship + 3 years	Archive

13.2 Archival strategy

- Time-partitioned tables (audit_logs, notification_messages, whatsapp_messages, workflow_executions, ai_tool_call_audit, ai_usage_logs, security_events, activity_events, outbox_events) are archived by partition: DETACH PARTITION → export to compressed Parquet/CSV in tenant-prefixed, encrypted object storage → verify checksum → DROP.
- Archive manifests (partition, row count, checksum, location) are recorded so restores are auditable; a P2 archive_manifests table can hold them.
- ERP records are archived by status, not deleted; Frappe's own archival is not relied upon.
- Legal hold flag on data_retention_policies suspends deletion/archival for a target or tenant.

13.3 Cleanup policies

- Nightly job (worker) removes expired idempotency keys, OTP challenges, expired auth sessions, expired ai_conversations and ai_pending_actions; each run writes a summary audit row.
- Retention role (separate from application role) owns partition DETACH/DROP; the application role cannot delete audit data.
- Orphan detection is report-only (reconciliation) — never destructive.

13.4 Backup policies

Store	Policy
PostgreSQL (production)	Managed instance with PITR (continuous WAL archiving), daily full backup, 35-day retention, encrypted; monthly tested restore into an isolated instance; cross-zone copy.
PostgreSQL (staging/dev)	Daily logical backup 7 days; dev is disposable, synthetic data only.
ERP (Frappe Cloud)	Frappe Cloud scheduled backups (daily) with retention per plan; pre-release manual backup before custom-app upgrade.
n8n	Database + credentials export backed up daily; workflows are also in Git.
Object storage / archives	Versioned, encrypted, lifecycle to infrequent-access after 90 days.
Secrets	Held in secret store; never in backups of application tables.

13.5 Recovery considerations

- Targets (proposed, to be agreed): RPO ≤ 15 minutes for PostgreSQL via PITR; RTO ≤ 4 hours in beta/early production.

- Cross-store consistency: PG and ERP restore independently. After any restore run reconciliation (ops.reconciliation_runs: erp_orphans, outbox_outcome, payment_pending) before re-enabling the outbox dispatcher; outbox replay is safe because consumers are idempotent.
- Rollback of releases uses backward-compatible migrations (expand → migrate → contract) so an API rollback does not require a database restore.
- Restore drills quarterly; evidence stored with test results (NFR-AVL-04).

14. Database Security

14.1 Row Level Security

All tenant-scoped tables enable and FORCE row-level security. A single policy pattern is applied by a migration helper (Appendix A):

```
CREATE POLICY tenant_isolation ON <schema>.<table>
  USING      (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::uuid)
  WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::uuid);
```

- Missing or empty setting yields no rows (fail closed).
- Global reference rows (tenant_id NULL: system roles, standard report definitions) use a second read-only policy: USING (tenant_id IS NULL).
- Platform-admin support access uses a separate role with time-boxed, audited grants — never BYPASSRLS on the application role.
- Automated tests: for every tenant table, insert as tenant A and assert zero visibility and failed writes as tenant B (runs in CI, Architecture §6.1).

14.2 Tenant Security

- Tenant guard (API) + RLS (PG) + ERP permission hooks + tenant-scoped AI retrieval: four independent layers.
- Composite foreign keys (tenant_id, parent_id) on parent-child tables prevent linking rows across tenants.
- Cross-tenant attempts return not-found style responses, insert audit.security_events (tenant_denied) and alert on repetition.
- Per-tenant credentials and templates; tenant-prefixed storage paths.

14.3 Encryption Strategy

Layer	Approach
In transit	TLS 1.2+ for app↔DB, app↔ERP, webhooks; HSTS at the edge.
At rest	Managed disk/instance encryption for PostgreSQL and backups; Frappe Cloud encryption for ERP; encrypted object storage.
Field-level (envelope)	Application-level AES-256-GCM with per-environment data key (KMS/secret store) for: recipient_address_enc, mfa_factors.secret_enc, and ERP Member.health_notes. Keys rotated; ciphertext carries key id.
Hashing	Passwords: argon2id. OTP, refresh tokens, idempotency request hashes, recipient dedupe hashes: SHA-256 (peppered where enumeration is a risk, e.g. phone hashes use HMAC-SHA256 with a server-side key).
Keys and secrets	Never in tables; only secret_ref/credential_ref pointers. Write-only in UI (B2B-SET-03).

14.4 Sensitive Data Protection

Data	Protection
Phone, email, DOB, emergency contact (PII)	Tenant-scoped RLS; masked in logs and AI prompts (PII minimisation); role-based field visibility in API; audit snapshots redacted.
Health notes (optional)	Envelope-encrypted; visible to Owner/Admin only; excluded from AI prompts, audit snapshots and exports by default.
Feedback comments	Raw text visible to Owner/Admin only; trainers see aggregates only.
Payment data	No card data stored (Razorpay hosted checkout). Only provider references and status; webhook headers redacted.
Notification recipients	Hash for dedupe; encrypted address retained only as long as retry needs it (nulled after 90 days).
AI conversations	Redacted content, short TTL, no training use; PII minimised before send to provider.
Non-production	Synthetic or anonymised data only (Architecture §13.1).

14.5 Access Controls

Database role	Privileges
app_rw (API/worker)	CONNECT; USAGE on schemas; SELECT/INSERT/UPDATE (and DELETE only where policy allows, e.g. sessions, OTP) on tenant tables; INSERT+SELECT only on append-only tables; no DDL; NOBYPASSRLS; not owner.
app_ro (reporting replica)	SELECT on reporting views and permitted tables via RLS; no PII columns without view.
migrator	DDL only; used by CI/CD; separate credentials; not usable from runtime.
retention_job	DETACH/DROP partitions and delete expired operational rows; no access to business data.
platform_support	Time-boxed, audited, read-only through support procedures; no BYPASSRLS; grants expire automatically.
n8n / agents	No database role at all — access only via API with service principals (PRD: AI and automation never access the database directly).

14.6 Audit Controls

- Append-only audit and AI audit enforced by privilege and trigger; cannot be altered by application role.
- pgaudit (or managed equivalent) logs DDL and privileged role activity to the platform log store.
- Every security-relevant denial writes audit.security_events; alerts on repeated tenant denials, invalid webhook signatures, backup failures and outbox backlog.
- Retention role actions (partition drops) are themselves logged to audit_logs with actor_type = system.

15. MVP Database Scope

Priorities are taken from the Module Specification (P0 = mandatory 5-day MVP) and PRD roadmap (P1 = Phase 1 additions, P2 = later). Categories: Core (platform, iam, config, ops, reporting, audit), ERP, AI, Agent, Automation. Cut order from the PRD applies: AI is cut first, never the core operational modules, tenant isolation or audit.

Priority	Core	ERP	AI	Agent	Automation	Total
P0	28	20	4	0	5	57
P1	10	0	7	0	2	19
P2	5	0	0	8	2	15

P0 tables — mandatory for the 5-Day MVP

Category	Tables
Core tables	platform.subscription_tiers, platform.tier_features, platform.tenants, platform.tenant_domains, platform.tenant_feature_flags, platform.studios, platform.branches, iam.users, iam.user_credentials, iam.auth_identities, iam.roles, iam.permissions, iam.role_permissions, iam.user_role_assignments, iam.permission_grants, iam.user_erp_links, iam.auth_sessions, iam.otp_challenges, config.tenant_branding, config.tenant_settings, config.event_categories, config.notification_templates, config.notification_rules, ops.outbox_events, ops.idempotency_keys, ops.webhook_inbox, audit.audit_logs, audit.security_events
ERP tables	ERP:Wellness Tenant Scope, ERP:Wellness Studio Scope, ERP:Wellness Branch Scope, ERP:Member, ERP:Membership Plan, ERP:Plan Class Entitlement, ERP:Membership, ERP:Class Type, ERP:Class Schedule, ERP:Class Session, ERP:Booking, ERP:Attendance Record, ERP:Attendance Correction, ERP:Event, ERP:Event Registration, ERP:Lead, ERP:Lead Activity, ERP:Payment Record, ERP:Wellness Staff, ERP:Staff Class Assignment
AI tables	ai.ai_sessions, ai.ai_conversations, ai.ai_pending_actions, ai.ai_tool_call_audit
Agent tables	—
Automation tables	notify.notification_messages, notify.whatsapp_messages, notify.notification_delivery_events, notify.workflow_executions, notify.admin_alerts

P1 tables — important (Phase 1 additions)

Category	Tables
Core tables	platform.tenant_erp_routes, iam.service_principals, config.faq_entries, ops.reconciliation_runs, ops.reconciliation_findings, reporting.report_definitions, reporting.kpi_snapshots, audit.change_history, audit.activity_events, audit.data_retention_policies
ERP tables	—
AI tables	ai.ai_recommendations, ai.ai_feedback_submissions, ai.ai_feedback_analysis, ai.ai_feedback_theme_summaries, ai.ai_usage_logs, ai.ai_prompt_versions, ai.ai_eval_runs
Agent tables	—
Automation tables	notify.communication_consents, notify.automation_audit_trails

P2 tables — future (schema reserved where noted)

Category	Tables
Core tables	platform.tenant_subscriptions, iam.mfa_factors, ops.erp_contract_versions, reporting.report_exports, audit.data_subject_requests
ERP tables	—
AI tables	—
Agent tables	agent.agent_registry, agent.agent_tenant_configs, agent.agent_tool_grants, agent.agent_tasks, agent.agent_executions, agent.agent_decisions, agent.agent_memory, agent.agent_logs
Automation tables	notify.email_logs, notify.suppression_list

15.1 MVP delivery notes

- Day 1 migration set: platform, iam, config (branding/settings/templates/rules/categories), ops (outbox, idempotency, webhook_inbox), audit (audit_logs, security_events), plus RLS helper, partition helpers and seed (permissions, system roles, tiers, default settings, default templates). ERP: scope mirrors, Member, Membership Plan, Class Type with atomic-method spikes S1–S3.
- Day 2–3: ERP Membership, Class Schedule/Session, Booking, Attendance (+Correction); ai_sessions/ai_conversations/ai_tool_call_audit/pending actions tables created alongside Assistant skeleton.
- Day 4: Payment Record, Lead, notify.* core tables with mock delivery, workflow_executions, admin_alerts; kpi_snapshots optional (dashboard may read ERP live in MVP).
- Day 5: Event, Event Registration; feedback/recommendation tables (P1) only if AI slice is not cut; tenant-isolation tests.
- If time slips (PRD cut order): drop P1 AI tables first, then feedback tables, then FAQ/enquiry storage; do not drop audit, RLS, outbox, idempotency or webhook inbox.

- Agent schema is created empty (P2) so no later redesign is needed; no Phase 1 code depends on it.

Appendix A — Implementation DDL Patterns

A.1 Bootstrap, tenant context and updated_at trigger

```
CREATE EXTENSION IF NOT EXISTS pgcrypto; CREATE EXTENSION IF NOT EXISTS citext; CREATE EXTENSION IF NOT EXISTS
pg_trgm;
CREATE SCHEMA IF NOT EXISTS platform; CREATE SCHEMA IF NOT EXISTS iam; CREATE SCHEMA IF NOT EXISTS config;
CREATE SCHEMA IF NOT EXISTS ops; CREATE SCHEMA IF NOT EXISTS notify; CREATE SCHEMA IF NOT EXISTS ai;
CREATE SCHEMA IF NOT EXISTS agent; CREATE SCHEMA IF NOT EXISTS audit; CREATE SCHEMA IF NOT EXISTS reporting;

CREATE FUNCTION platform.current_tenant() RETURNS uuid LANGUAGE sql STABLE AS
$$ SELECT NULLIF(current_setting('app.tenant_id', true), '')::uuid $$;

CREATE FUNCTION platform.set_updated_at() RETURNS trigger LANGUAGE plpgsql AS
$$ BEGIN NEW.updated_at := now(); RETURN NEW; END $$;

-- API, at start of every request transaction:
--   BEGIN; SELECT set_config('app.tenant_id', $1, true); ...
```

A.2 RLS helper applied by migration to every tenant table

```
CREATE PROCEDURE platform.enable_tenant_rls(p_schema text, p_table text) LANGUAGE plpgsql AS $$
BEGIN
  EXECUTE format('ALTER TABLE %I.%I ENABLE ROW LEVEL SECURITY', p_schema, p_table);
  EXECUTE format('ALTER TABLE %I.%I FORCE ROW LEVEL SECURITY',  p_schema, p_table);
  EXECUTE format('CREATE POLICY tenant_isolation ON %I.%I
      USING (tenant_id = platform.current_tenant())
      WITH CHECK (tenant_id = platform.current_tenant())', p_schema, p_table);
END $$;
```

A.3 Example: membership-independent core tables (tenants, users)

```
CREATE TABLE platform.tenants (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  code varchar(40) NOT NULL UNIQUE CHECK (code ~ '^[a-z0-9][a-z0-9-]{1,38}$'),
  legal_name varchar(200) NOT NULL, display_name varchar(120) NOT NULL,
  tier_id uuid NOT NULL REFERENCES platform.subscription_tiers(id),
  status varchar(20) NOT NULL DEFAULT 'active' CHECK (status IN ('onboarding','active','suspended','closed')),
  timezone varchar(60) NOT NULL DEFAULT 'Asia/Kolkata', currency char(3) NOT NULL DEFAULT 'INR',
  locale varchar(10) NOT NULL DEFAULT 'en-IN', erp_company varchar(140),
  data_region varchar(20) NOT NULL DEFAULT 'in-west', closed_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now());

CREATE TABLE iam.users (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES platform.tenants(id),
  user_type varchar(10) NOT NULL CHECK (user_type IN ('staff','member','platform')),
  email citext, phone_e164 varchar(16), display_name varchar(160) NOT NULL,
  status varchar(20) NOT NULL DEFAULT 'active' CHECK (status IN ('invited','active','locked','deactivated')),
  must_reset_password boolean NOT NULL DEFAULT false, last_login_at timestamptz, deactivated_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now(),
  CHECK (email IS NOT NULL OR phone_e164 IS NOT NULL));
CREATE UNIQUE INDEX uq_users_tenant_email ON iam.users (tenant_id, email) WHERE email IS NOT NULL;
CREATE UNIQUE INDEX uq_users_tenant_phone ON iam.users (tenant_id, phone_e164) WHERE phone_e164 IS NOT NULL;
CALL platform.enable_tenant_rls('iam','users');
```

A.4 Partitioned, append-only audit table

```
CREATE TABLE audit.audit_logs (
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL, occurred_at timestamptz NOT NULL DEFAULT now(),
  actor_type varchar(12) NOT NULL CHECK (actor_type IN ('human','automation','ai','agent','system')),
  actor_id uuid, action varchar(60) NOT NULL, resource_type varchar(40) NOT NULL, resource_ref varchar(140) NOT NULL,
  before_data jsonb, after_data jsonb, reason text, request_context jsonb NOT NULL DEFAULT '{}',
  correlation_id uuid NOT NULL, prev_hash char(64), row_hash char(64),
  PRIMARY KEY (id, occurred_at)) PARTITION BY RANGE (occurred_at);
CREATE INDEX ix_audit_tenant_time ON audit.audit_logs (tenant_id, occurred_at DESC);
-- monthly partitions created by a scheduled job (pg_partman or worker) 3 months ahead

REVOKE UPDATE, DELETE, TRUNCATE ON audit.audit_logs FROM app_rw;
CREATE FUNCTION audit.forbid_mutation() RETURNS trigger LANGUAGE plpgsql AS
$$ BEGIN RAISE EXCEPTION 'audit tables are append-only'; END $$;
CREATE TRIGGER trg_audit_immutable BEFORE UPDATE OR DELETE ON audit.audit_logs
  FOR EACH ROW EXECUTE FUNCTION audit.forbid_mutation();
CALL platform.enable_tenant_rls('audit','audit_logs');
```

A.5 Outbox insert and dispatcher claim (transactional, idempotent)

```
-- Same transaction as audit insert, after the ERP call succeeded
INSERT INTO ops.outbox_events (tenant_id, event_type, subject_type, subject_ref, payload, correlation_id)
VALUES ($1,'booking.confirmed','Booking',$2,$3,$4);

-- Dispatcher (worker) claims a batch without blocking other workers
WITH c AS (
  SELECT id FROM ops.outbox_events
  WHERE status IN ('pending','failed') AND next_attempt_at <= now()
  ORDER BY next_attempt_at FOR UPDATE SKIP LOCKED LIMIT 50)
UPDATE ops.outbox_events o SET status='dispatching', attempts=attempts+1
FROM c WHERE o.id=c.id RETURNING o.*;
-- on success: status='delivered', delivered_at=now()
```

```
-- on failure: status='failed', next_attempt_at=now()+backoff(attempts); after N attempts status='dead' +
admin_alerts row
```

A.6 Notification idempotency and consent check

```
INSERT INTO notify.notification_messages
  (tenant_id, outbox_event_id, event_type, template_id, channel, recipient_kind, recipient_erp_ref,
   recipient_address_hash, dedupe_key, mode, status, correlation_id)
VALUES ($1,$2,$3,$4,'whatsapp','member',$5,$6, $2::text||':'||$5||':'||$4::text, $7,'queued',$8)
ON CONFLICT (tenant_id, dedupe_key) DO NOTHING RETURNING id;   -- no row = duplicate, do not send
```

A.7 Pending-action execution guard (AI)

```
UPDATE ai.ai_pending_actions SET status='confirmed', decided_at=now()
WHERE id=$1 AND tenant_id=platform.current_tenant() AND requested_by=$2
  AND status='proposed' AND expires_at > now() AND arguments_hash = encode(digest($3::text,'sha256'),'hex')
RETURNING arguments;   -- zero rows = expired, already used, wrong user, or arguments drifted
```

Appendix B — Migration Order and Team Ownership

B.1 Migration order (per environment)

Step	Migration	Depends on
1	000_bootstrap: extensions, schemas, helper functions, roles/grants	—
2	010_platform: tiers, tier_features, tenants, tenant_domains, feature_flags, studios, branches	1
3	020_iam: users … permissions, assignments, grants, sessions, otp, user_erp_links, service_principals	2
4	030_config: branding, settings, event_categories, templates, rules (+FAQ P1)	2
5	040_ops: outbox_events (partitioned), idempotency_keys, webhook_inbox (+reconciliation P1)	2
6	050_audit: audit_logs (partitioned), security_events (+change_history, activity_events, retention policies P1)	2
7	060_notify: notification_messages, whatsapp_messages, delivery events, workflow_executions, admin_alerts (+consents P1)	4,5
8	070_ai: sessions, conversations, pending actions, tool-call audit (+P1 tables)	3,6
9	080_reporting: report_definitions, kpi_snapshots	2
10	090_agent: all agent tables (empty; P2)	3,8
11	099_seed: permissions catalogue, system roles + role_permissions, tiers/features, default settings/templates/rules, retention defaults	all
12	ERP: bench migrate of wellness_studio (patches/fixtures for DocTypes, scope mirrors, seed class types)	onboarding routine

B.2 Schema ownership (proposal, aligned to Architecture §14.7)

Owner	Schemas / DocTypes	Notes
Tahir (solution lead)	bootstrap, platform, iam, ops, audit; RLS helpers; ERP adapter data contracts	Foundation; reviews every migration PR
Harish	notify (with n8n workflows/worker), config.notification_*, reporting	Outbox consumer side, dedupe, delivery status
Shreya	ai (gateway, feedback, recommendations, usage, eval), agent, config.faq_entries	Agent schema stays reserved until Phase 2
Harshini	ai_sessions, ai_conversations, ai_pending_actions, ai_tool_call_audit (Admin Assistant flow)	Coordinates with Shreya on shared audit shape
Member 5 (ERP)	All ERP DocTypes (Part B), atomic methods, ERP indexes, scope mirrors	Tenant permission hooks; spike S1–S3
Member 6 (Frontend)	Consumers only; owns config.tenant_branding and config.event_categories usage contract with UI	No direct DB access

B.3 Rules for parallel development

- One migration folder per schema; filenames prefixed by numeric order; no cross-schema DDL in a single migration except FK additions in a follow-up file.
- Cross-schema FKs point only “downwards” (notify → ops/config/platform; ai → iam/platform); never platform → other schemas.
- Every migration is backward-compatible (expand → migrate → contract) and reviewed by the schema owner plus Tahir.
- Seed data is idempotent (INSERT … ON CONFLICT DO NOTHING) and environment-independent; demo data is a separate script (synthetic only).
- Prisma schema is generated/maintained per schema; RLS, partitions, triggers and policies live in raw SQL migrations that Prisma does not manage (documented in each folder).